# Reporte

## Producto Integrador de Aprendizaje · Eventos Nova

Implementación de un sistema para contratación, coordinación de proveedores, cambios, pagos, rentabilidad y cierre de eventos.

### Datos de identificación

| Integrante | Matrícula |
|---|---|
| Osmar Abdi Sanchez Santiago | 2226699 |
| Omar Tadeo Gasca Deciga | 2226708 |
| Gabriela Alejandra Fernández Briones | 2226870 |

| Dato | Valor |
|---|---|
| Curso | Componentes de Lógica de Negocios |
| Profesor | Miguel Ángel Gómez Marroquín |
| Actividad | Producto Integrador de Aprendizaje (PIA) |
| Equipo y empresa | Equipo 16 · Eventos Nova |
| Institución | FIME · Universidad Autónoma de Nuevo León |
| Fecha del corte documental | 06 de noviembre de 2026 |
| Repositorio | https://github.com/OmarHub-star/PIA-Eventos-Nova |
| Sistema desplegado | https://eventos-nova-pia-cv0t.onrender.com/ |

# Objetivo, antecedentes y procedimiento

## Objetivo

Integrar los productos de las Unidades 2 y 3 en una solución de software funcional que aplique DDD, Specification, State Machine, API REST, persistencia, pruebas automatizadas y un entorno de ejecución reproducible. La entrega se organiza conforme a la rúbrica oficial de noviembre de 2026.

## Antecedentes

Eventos Nova organiza bodas, graduaciones y eventos empresariales. El proceso comercial involucra clientes, coordinación, administración y proveedores, con información sensible al estado de una propuesta, al pago del contrato y al tiempo disponible antes del evento. Los intercambios fragmentados de mensajes generan incertidumbre sobre la versión vigente, las autorizaciones y las obligaciones pendientes.

## Solicitud

Implementar el negocio descrito por el BRD, historias, criterios Gherkin y BPMN de Eventos Nova. La arquitectura y el código deben guardar correspondencia con las mismas reglas, y las pruebas deben demostrar transiciones permitidas y operaciones rechazadas.

## Procedimiento

- Refinamiento del análisis y de diez reglas de negocio trazables.
- Modelado de quince historias de usuario, Gherkin, BPMN y glosario.
- Definición de agregados, Value Objects, servicios, estados, patrones y ADR.
- Implementación de casos de uso, repositorios SQL y endpoints FastAPI.
- Pruebas unitarias, integración, CSV, calidad estática y cobertura.
- Preparación de CI/CD, Docker Compose, demostración y despliegue.

## Resumen del sistema para el negocio

- La propuesta se versiona y conserva su historial al revisarse.
- La contratación se confirma al registrar referencia contractual y anticipo acumulado de al menos 30%.
- Las solicitudes de proveedor aceptan respuestas durante menos de 48 horas; pueden vencer o requerir Backup.
- Los cambios a menos de 15 días incorporan recargo y autorización; los cambios a tiempo no cobran ese cargo.
- Los servicios adicionales se pagan antes de prestarse; el cierre exige liquidación y acta.
- Cada actor dispone de acciones autorizadas y el sistema conserva auditoría de eventos de dominio.

| Acceso | Dirección o condición |
|---|---|
| Repositorio | https://github.com/OmarHub-star/PIA-Eventos-Nova |
| Swagger local | http://localhost:8000/docs |
| Swagger público | Disponible en https://eventos-nova-pia-cv0t.onrender.com/docs |

# 1. Análisis del negocio - BRD

Versión 1.0. Fecha: 06/11/2026. Base: BRD de U2 (04/09/2026) e informe de U3 (17/10/2026). Se mantienen los identificadores originales para trazabilidad. Las decisiones nuevas de implementación se distinguen en los ADR.

## 1. Propósito y modelo de negocio

Eventos Nova organiza bodas, graduaciones y eventos empresariales. Coordina servicios de salón, alimentos, mobiliario, música, decoración y fotografía contratados con proveedores. Necesita administrar desde la solicitud del cliente hasta la liquidación y cierre, con una única propuesta vigente, responsables identificables y consecuencias económicas trazables.

## 2. Situación actual y problemas

El Cliente comunica fecha, aforo y tipo de evento. Coordinación cotiza, recibe aceptación y formaliza el contrato. Administración cobra el anticipo. Coordinación solicita disponibilidad y realiza seguimiento. Los cambios por WhatsApp dispersan la información.

| ID | Problema | Tratamiento en la solución |
|---|---|---|
| P01 | Cambios no llegan a todos | Registro de avisos internos con destinatarios |
| P02 | Versiones distintas de una propuesta | Historial de snapshots y versión vigente |
| P03 | Proveedores trabajan con información anterior | Evento CambioAplicado con versión y destinatarios |
| P04 | Cambios autorizados no cobrados | Incremento, cargo y autorización ligados al evento |
| P05 | Seguimiento manual | Estados y temporizador de vencimientos |

## 3. Objetivos e indicadores

OB01: reducir a 0% los incidentes por desincronización atribuida a WhatsApp. OB02: lograr que 90% de proveedores responda en 24 horas. OB03: aumentar 15% el margen global por reducción de fugas de ingresos. OB04: conservar trazabilidad de solicitudes, aprobaciones y ajustes.

Son **metas del negocio**, no resultados comprobados del prototipo. La demo demuestra mecanismos de control, no mide impacto comercial real ni disponibilidad operativa real.

## 4. Stakeholders y permisos

Cliente: proporciona requisitos, acepta/rechaza propuesta, registra la referencia de contrato y solicita cambios de su evento. Coordinador: prepara y versiona propuestas, asigna proveedores, cotiza cambios, registra adicionales, inicia/finaliza y cierra. Administración: registra pagos, decide excepciones y consulta rentabilidad. Proveedor: responde y cancela sus propias solicitudes. Sistema: identifica solicitudes vencidas y registra alertas.

Cada token identifica una persona o actor concreto. Cliente solo accede a sus eventos. Proveedor solo recibe sus solicitudes y no consulta pagos del cliente. Las decisiones financieras se reservan a Administración.

## 5. Alcance

Incluye alta del evento, propuesta versionada, aceptación/rechazo, contrato, anticipo, coordinación y sustitución de proveedores, plazos, cambios y autorizaciones, adicionales y cobros, rentabilidad por evento, ejecución, liquidación y cierre con acta. La interfaz de demostración es Swagger UI.

Fuera de alcance: contabilidad general, nómina, recursos humanos, marketing, diseño creativo integral, rutas de transporte, operaciones sin un evento Nova y procesamiento bancario real. Las notificaciones son internas. La firma consiste en el registro autenticado de referencia contractual, sin certificación de firma electrónica.

## 6. Necesidades

| ID | Necesidad | Prioridad |
|---|---|---|
| BN01 | Comunicación centralizada | Alta |
| BN02 | Propuesta vigente e historial | Alta |
| BN03 | Avisar cambios relevantes | Alta |
| BN04 | Registrar solicitante y autorizador | Alta |
| BN05 | Impacto comercial y rentabilidad | Alta |
| BN06 | Solicitudes, decisiones y plazos de proveedores | Alta |

| ID | Necesidad | Prioridad |
|---|---|---|
| BN07 | Anticipos, pagos y saldos | Media |
| BN08 | Proveedores Backup para contingencias | Media |

## 7. Reglas verificables

| ID | Regla refinada | Evidencia |
|---|---|---|
| BR01 | Contrato firmado y anticipo acumulado >=30% del monto firmado para confirmar. No se generan reembolsos del anticipo al cancelar. | Event.sign/pay/cancel |
| BR02 | Respuesta antes de `sent_at` + 48h. En el límite exacto ya está vencida. Temporizador cada 60 s registra la alerta. | SupplierRequest.respond/expire |
| BR03 | Cambios a 15 días naturales o más del evento sin penalización | ChangePolicy.evaluate |
| BR04 | Cambios a 14 días o menos: 15% del incremento, pendientes de decisión de Administración | request_change/apply_change/reject_change |
| BR05 | Registrar el adicional antes de prestarlo | add_extra/deliver_extra |
| BR06 | Ante cancelación/rechazo/vencimiento, registrar sustitución mediante identificador de solicitud anterior y proveedor Backup activo | request_supplier |
| BR07 | Una versión vigente y conservación de todas las anteriores | Proposal.revise, transacción conjunta |
| BR08 | Pago identificado por terminal o transferencia antes de prestar el adicional | pay_extra/deliver_extra |

| ID | Regla refinada | Evidencia |
|---|---|---|
| BR09 | Cierre con saldo cero, contrato, cambios resueltos, adicionales prestados y acta | CAN_CLOSE y Event.close |
| BR10 | Registrar responsable, instante, condición anterior/nueva y decisión | changes, versions, domain_events |

Los límites de catálogo se interpretan como capacidad máxima por línea de cotización. El CSV no define inventario compartido ni un aforo total de recinto. Para disponibilidad de proveedores se impide reservar al mismo proveedor dos veces en una fecha mediante restricción única en DB.

## 8. Requisitos no funcionales

RNF01: auditoría persistente consultable. RNF02: total vigente consistente con versión. RNF03: autenticación y permisos por identidad. RNF04: health check y despliegue reproducible; no se promete SLA. RNF05: errores y estados comprensibles. RNF06: no cerrar con datos críticos incompletos. RNF07: alertas registradas con destinatarios. RNF08: rollback de evento, propuesta y eventos de dominio si falla la transacción.

Calidad PIA: cobertura >=80% de `src`, pruebas unitarias e integración, estilos verificados, OpenAPI, configuración por entorno, Docker Compose y CI/CD. El estado realmente verificado se registra en `evidencias/RESUMEN.md`.

## 9. Proceso TO-BE y excepciones

La propuesta inicia en BORRADOR. Coordinación la envía y el Cliente acepta o rechaza. Tras rechazo se prepara nueva versión. La aceptación habilita contrato y pagos; alcanzar el anticipo confirma el evento. Coordinación asigna servicios. Cada proveedor responde dentro del plazo o vence y se inicia contingencia. Los cambios preservan el historial y la autorización. El evento se ejecuta en su fecha con proveedores confirmados y sin cambios pendientes. Se resuelven adicionales, se finaliza, se liquida si falta saldo y se firma el acta de cierre.

Excepciones modeladas: proveedor cancela, rechazo, vencimiento, duplicidad de reserva, aumento de invitados, cambio de horario/menú, recargo tardío, cambio rechazado, adicional sin pago, saldo pendiente y transición repetida o inválida.

## 10. Ambigüedades y supuestos

El informe U3 §1.4 registra H1-H5 como validados por el equipo aunque el ADR-005 aún figura propuesto. Se aplican: liquidación antes o después del evento; día 15 inclusive sin cargo; cálculo automático y autorización humana; 15% sobre incremento; adicional sin pago no prestado. El PIA implementa también las extensiones de ejecución y cierre.

Nuevas decisiones: cambios del Cliente requieren revisión comercial incluso dentro de plazo; los proveedores de demo son datos complementarios; la fecha de evento para operaciones CSV se deriva de solicitud +60 días; el precio del cambio lo cotiza manualmente un responsable. No existe fórmula de precio por invitado en las fuentes. La aprobación no acredita una validación por el profesor ni un stakeholder externo.

## 11. Éxito y aceptación

Cada US01-US15 debe tener criterio Gherkin, componente y prueba identificables. La demostración debe mostrar un flujo completo y rechazos verificables. Se acepta la entrega técnica cuando las pruebas, cobertura, documentación, repositorio y despliegue coinciden con la versión presentada. Las metas OB01-OB03 requieren medición en operación real.

## 12. Conclusión

La lógica central conserva una versión coherente del evento y permite relacionar autorización, cobro y prestación del servicio. El prototipo hace verificables las reglas del BRD y mantiene explícitas las decisiones que las fuentes no definen, facilitando su revisión por el equipo.

# 2. Historias de usuario y criterios Gherkin

Se incluyen US01-US15 con sus escenarios completos y su relación con reglas, casos de uso, endpoints y pruebas.

Se conservan US01-US15 de U2 y se refinan conforme a H1-H5 de U3. Cada criterio tiene prueba pytest identificada en TRAZABILIDAD.csv. El archivo `aceptacion.feature` conserva el formato Gherkin; las pruebas ejecutables son las de pytest.

## US01

Como Coordinador, quiero registrar datos base y servicios, para preparar una propuesta V1.0.

Reglas: BN02, BR07. Componente: `Event, Proposal.revise`. API: `POST /events`. Prueba: `test_US07_revision_history_and_duplicate_accept`.

```
@US01
Escenario: US01 criterio 1
  Dado un cliente activo, fecha futura y servicios habilitados
  Cuando el Coordinador registra el evento
  Entonces se conserva el evento BORRADOR y la propuesta V1.0 con el total decimal
@US01
Escenario: US01 criterio 2
  Dado un aforo de cero
  Cuando se intenta registrar el evento
  Entonces la API responde 422 y no persiste el alta
```

## US02

Como Cliente, quiero aceptar la propuesta enviada, para iniciar la contratación.

Reglas: BR07. Componente: `Proposal.accept, Event.accept`. API: `POST /events/{id}/accept`. Prueba: `test_US07_revision_history_and_duplicate_accept`.

```
@US02
Escenario: US02 criterio 1
  Dado una propuesta ENVIADA del Cliente autenticado
  Cuando el Cliente acepta
  Entonces la propuesta queda ACEPTADA y el evento ACEPTADO
@US02
Escenario: US02 criterio 2
  Dado una propuesta ya aceptada
  Cuando el Cliente vuelve a aceptar
  Entonces la API responde 409
```

## US03

Como Administración, quiero registrar el anticipo del 30%, para confirmar el evento.

Reglas: BR01. Componente: `Event.pay`. API: `POST /events/{id}/pay`. Prueba: `test_BR01_30_percent_boundary`.

```
@US03
Escenario: US03 criterio 1
  Dado un contrato firmado por 1000.00 MXN
  Cuando Administración registra 300.00 MXN
  Entonces el evento pasa a CONFIRMADO
@US03
Escenario: US03 criterio 2
  Dado un contrato firmado por 1000.00 MXN
  Cuando se registran 299.99 MXN
  Entonces el evento permanece ACEPTADO
```

## US04

Como Coordinador, quiero solicitar disponibilidad de servicios, para coordinar proveedores.

Reglas: BN03, BN06. Componente: `SupplierRequest, NovaService.request_supplier`. API: `POST /events/{id}/requests`. Prueba: `test_US01_US15_full_business_cycle`.

```
@US04
Escenario: US04 criterio 1
  Dado un evento CONFIRMADO y proveedor activo
  Cuando Coordinación solicita el servicio
  Entonces se crea solicitud PENDIENTE y evento con destinatario y vencimiento
@US04
Escenario: US04 criterio 2
  Dado un evento BORRADOR
  Cuando Coordinación intenta asignar proveedor
  Entonces la API responde 409
```

## US05

Como Proveedor, quiero aceptar o rechazar mi solicitud dentro del plazo, para resolver mi participación.

Reglas: BR02. Componente: `SupplierRequest.respond`. API: `POST /requests/{id}/respond`. Prueba: `test_BR02_continuous_hours`.

```
@US05
Escenario: US05 criterio 1
  Dado una solicitud PENDIENTE de mi identidad y menos de 48 horas
  Cuando respondo que acepto
  Entonces la solicitud pasa a CONFIRMADO y se reserva la fecha
@US05
Escenario: US05 criterio 2
  Dado 48 horas exactas desde el envío
  Cuando intento aceptar
  Entonces la API responde 422 por vencimiento
```

## US06

Como Sistema, quiero detectar solicitudes sin respuesta, para avisar a Coordinación.

Reglas: BR02, RNF07. Componente: `SupplierRequest.expire, expiry_loop`. API: `Temporizador y POST /requests/expire`. Prueba: `test_US06_48_hours_and_expiration_idempotence`.

```
@US06
Escenario: US06 criterio 1
  Dado una solicitud PENDIENTE con 48 horas cumplidas
  Cuando el temporizador ejecuta su revisión
  Entonces la solicitud pasa a EXPIRADO y queda alerta interna a Coordinación
@US06
Escenario: US06 criterio 2
  Dado una solicitud ya expirada
  Cuando se revisa nuevamente
  Entonces no se duplica la alerta
```

## US07

Como Coordinador, quiero generar nuevas versiones de propuesta, para conservar el historial.

Reglas: BR07, BR10. Componente: `Proposal.revise`. API: `POST /events/{id}/revise`. Prueba: `test_US07_revision_history_and_duplicate_accept`.

```
@US07
Escenario: US07 criterio 1
  Dado una propuesta BORRADOR V1.0
  Cuando Coordinación cambia servicios con motivo
  Entonces se agrega V1.1 sin modificar V1.0
@US07
Escenario: US07 criterio 2
  Dado una propuesta ENVIADA
  Cuando se intenta editar directamente
  Entonces se rechaza la modificación hasta resolver la propuesta
```

## US08

Como Proveedor, quiero recibir el aviso interno de cambios aprobados, para usar la información vigente.

Reglas: RNF07, RNF08. Componente: `CambioAplicado, NovaService.apply_change`. API: `POST /events/{id}/change y GET /events/{id}/audit`. Prueba: `test_US08_US10_changes_authorization_and_notification`.

```
@US08
Escenario: US08 criterio 1
  Dado un proveedor asignado al evento
  Cuando se aplica un cambio de horario o menú
  Entonces CambioAplicado conserva la nueva versión y el proveedor como destinatario
@US08
Escenario: US08 criterio 2
  Dado un error antes del commit
  Cuando falla la actualización de propuesta
  Entonces evento, propuesta y notificación se revierten juntos
```

## US09

Como Cliente, quiero solicitar cambios de aforo con anticipación, para actualizar la cotización sin recargo.

Reglas: BR03. Componente: `ChangePolicy.evaluate, Event.request_change`. API: `POST /events/{id}/change`. Prueba: `test_client_change_waits_for_commercial_review`.

```
@US09
Escenario: US09 criterio 1
  Dado un evento a 15 días naturales o más
  Cuando el Cliente solicita aumentar invitados
  Entonces se calcula recargo 0.00 y espera revisión comercial
@US09
Escenario: US09 criterio 2
  Dado una cotización revisada de un cambio dentro de plazo
  Cuando Administración la autoriza
  Entonces se actualizan aforo y versión sin penalización
```

## US10

Como Administración, quiero autorizar cambios tardíos con recargo, para proteger el margen.

Reglas: BR04. Componente: `ChangePolicy, Event.apply_change/reject_change`. API: `POST /events/{id}/approve y reject_change`. Prueba: `test_US08_US10_changes_authorization_and_notification`.

```
@US10
Escenario: US10 criterio 1
  Dado un evento a 14 días con incremento de 100.00 MXN
  Cuando se solicita el cambio
  Entonces queda PENDIENTE con cargo calculado de 15.00 MXN sin cambiar aún el total
@US10
Escenario: US10 criterio 2
  Dado un cambio tardío pendiente
  Cuando Administración autoriza
  Entonces se aplican incremento y cargo con responsable y nueva versión
```

## US11

Como Coordinador, quiero gestionar una sustitución Backup, para mantener continuidad tras una cancelación.

Reglas: BR06, BN08. Componente: `SupplierRequest.cancel, request_supplier`. API: `POST /requests/{id}/cancel y POST /events/{id}/requests`. Prueba: `test_US11_backup_replacement_and_booking`.

```
@US11
Escenario: US11 criterio 1
  Dado un proveedor CONFIRMADO
  Cuando el Proveedor cancela con motivo
  Entonces se libera su reserva y se devuelve la lista Backup activa
@US11
Escenario: US11 criterio 2
  Dado una solicitud cancelada y proveedor Backup activo
  Cuando Coordinación registra su sustitución
  Entonces la nueva solicitud conserva el identificador de la anterior
```

## US12

Como Administración, quiero consultar ingresos y costo de proveedores, para evaluar el margen por evento.

Reglas: BN05. Componente: `NovaService.profitability`. API: `GET /events/{id}/profitability`. Prueba: `test_US01_US15_full_business_cycle`.

```
@US12
Escenario: US12 criterio 1
  Dado un total de 5990.00 y proveedores por 1000.00 MXN
  Cuando Administración consulta rentabilidad
  Entonces obtiene margen de 4990.00 MXN e importes cobrados por separado
@US12
Escenario: US12 criterio 2
  Dado un Cliente autenticado
  Cuando intenta consultar la rentabilidad
  Entonces la API responde 403
```

## US13

Como Coordinador, quiero registrar y cobrar servicios adicionales, para evitar prestación sin control comercial.

Reglas: BR05, BR08. Componente: `Event.add_extra/pay_extra/deliver_extra`. API: `POST /events/{id}/extra, pay_extra, deliver_extra`. Prueba: `test_BR05_BR08_extra_requires_payment_and_trace`.

```
@US13
Escenario: US13 criterio 1
  Dado un evento EN_EJECUCION y adicional registrado
  Cuando Administración registra su pago específico por terminal
  Entonces el adicional queda pagado y puede prestarse
@US13
Escenario: US13 criterio 2
  Dado un adicional sin pago
  Cuando Coordinación intenta marcarlo prestado
  Entonces la API responde 422
```

## US14

Como Administración, quiero liquidar el saldo del evento, para resolver la obligación financiera.

Reglas: BR09, H1 U3. Componente: `Event.pay, balance`. API: `POST /events/{id}/pay`. Prueba: `test_post_event_liquidation_follows_U3_H1`.

```
@US14
Escenario: US14 criterio 1
  Dado un evento FINALIZADO con saldo de 4123.00 MXN
  Cuando Administración registra 4123.00 MXN
  Entonces el saldo queda en 0.00 sin duplicar el anticipo
@US14
Escenario: US14 criterio 2
  Dado un saldo de 4123.00 MXN
```

```
    Cuando se intenta registrar un pago de 4123.01 MXN
    Entonces la API responde 422
```

## US15

Como Coordinador, quiero cerrar el evento con acta, para concluir su ciclo.

Reglas: BR09, BR10. Componente: `CAN_CLOSE, Event.close`. API: `POST /events/{id}/close`. Prueba: `test_US01_US15_full_business_cycle`.

```
@US15
Escenario: US15 criterio 1
  Dado un evento FINALIZADO sin saldo ni pendientes
  Cuando Coordinación registra el acta de cierre
  Entonces el estado queda CERRADO con fecha y responsable
@US15
Escenario: US15 criterio 2
  Dado un evento con saldo pendiente
  Cuando se intenta cerrar
  Entonces la API responde 422 y conserva su estado
```

## 3. BPMN y procesos de excepción

El archivo `architecture/eventos_nova.bpmn` es BPMN 2.0 editable con cuatro carriles: Cliente, Coordinación, Administración y Proveedor. Se abre con un modelador BPMN compatible. Contiene proceso principal, rechazo de propuesta, validación de anticipo, solicitudes, temporizador de 48 horas, rechazo/vencimiento, contingencia Backup, cambios opcionales, adicionales, liquidación y cierre.

El temporizador `PT48H` es un boundary event que interrumpe la espera de respuesta. En el código corresponde a `SupplierRequest.expire` y al adaptador `expiry_loop`. Los gateways de aceptación, anticipo, proveedor y saldo conservan alternativas explícitas. `Gestionar cambios` y `Resolver adicionales` son subprocesos colapsados en el plano principal y contienen sus nodos y secuencias en el XML. Los archivos SVG complementarios permiten consultar su detalle. Los data objects representan propuesta y contrato.

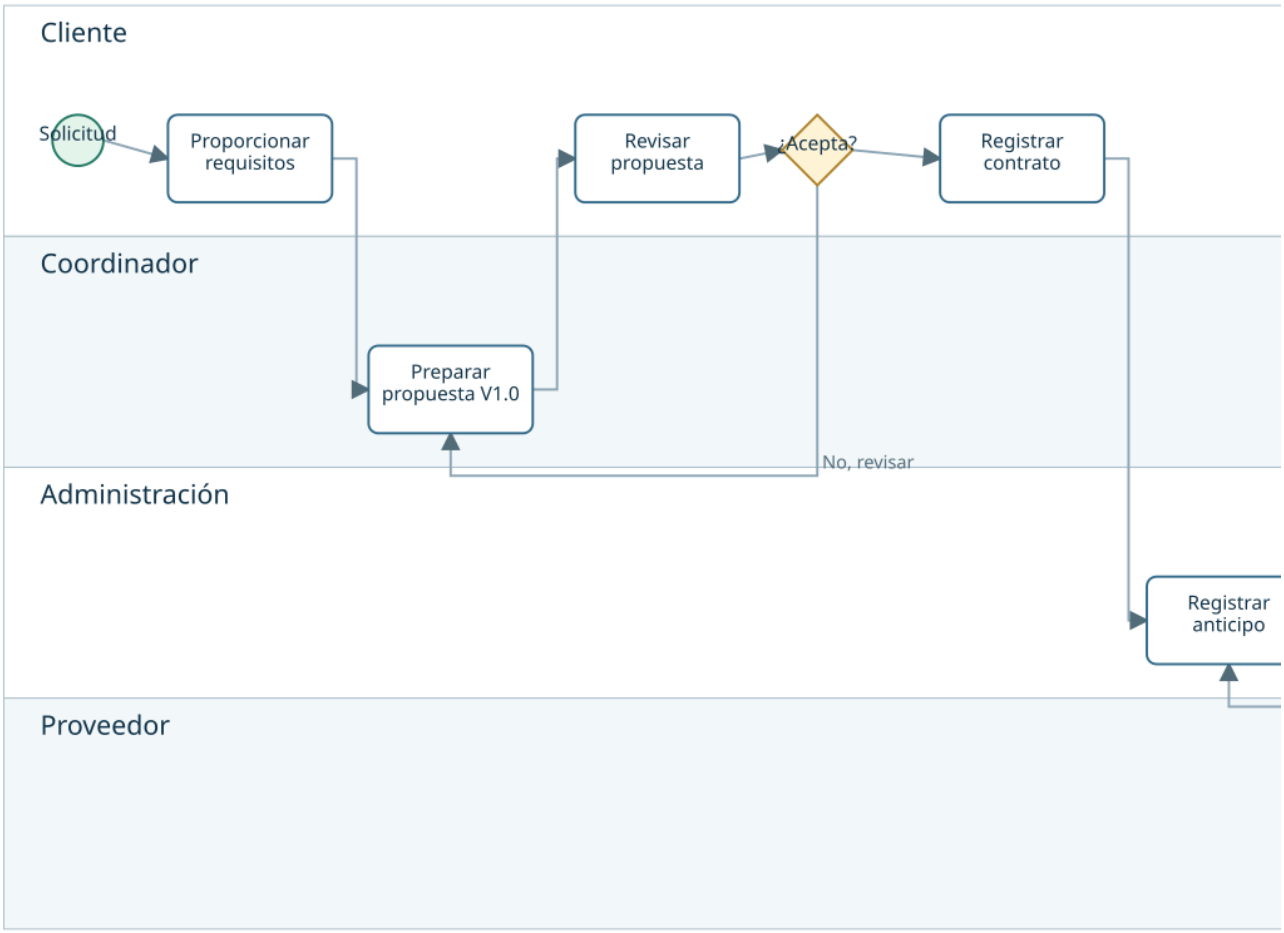
El BPMN describe el proceso de negocio, no un motor de ejecución. `isExecutable=false` indica que FastAPI y el modelo de dominio implementan sus reglas. No se afirma que el XML se despliegue en un motor BPMN.

Refinamientos frente a U2: rechazo vuelve a cotización, borde temporal explícito, cambio tardío sujeto a autorización, historial de versión, pago del adicional previo a prestación y liquidación general compatible con H1 de U3. En una operación real las actividades no se bloquean esperando un correo: el estado persiste entre peticiones y el proveedor responde de forma independiente.

La integridad de IDs y referencias de secuencia fue verificada localmente. La rúbrica debe evaluarse con el XML, las historias, el código y las pruebas de la matriz. El PDF BPMN original de U2 también se conserva en `docs/fuentes`.

Para conservar la legibilidad, el diagrama general se reproduce en tres tramos. El XML BPMN 2.0 completo, editable y con los cuatro carriles se incluye en docs/architecture/eventos_nova.bpmn.

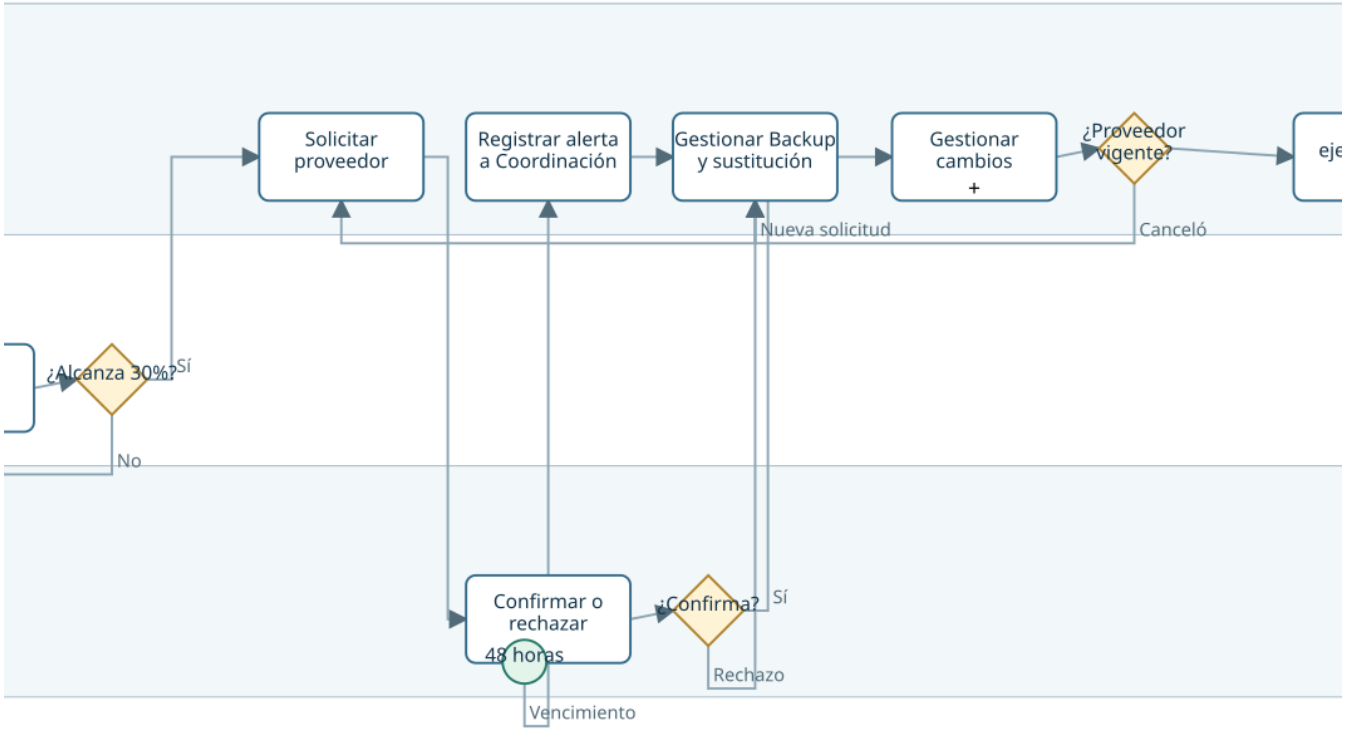
## BPMN - segmento 1 de 3




Fragmento del modelo BPMN del proceso integral.

El modelo contempla gateways, temporizador de 48 horas, rechazo de propuesta, contingencia, cambios y cierre.

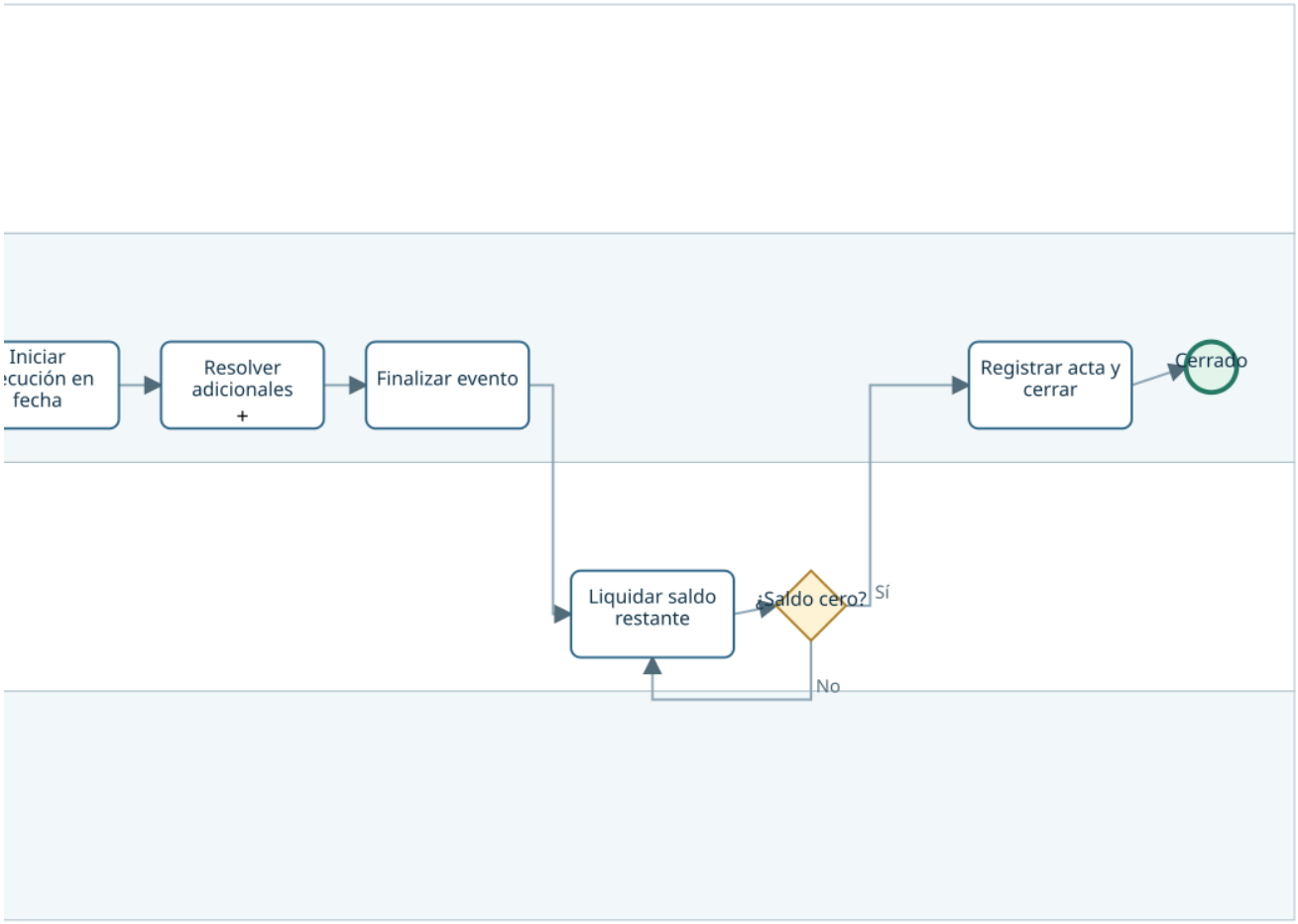
## BPMN - segmento 2 de 3




Fragmento del modelo BPMN del proceso integral.

El modelo contempla gateways, temporizador de 48 horas, rechazo de propuesta, contingencia, cambios y cierre.

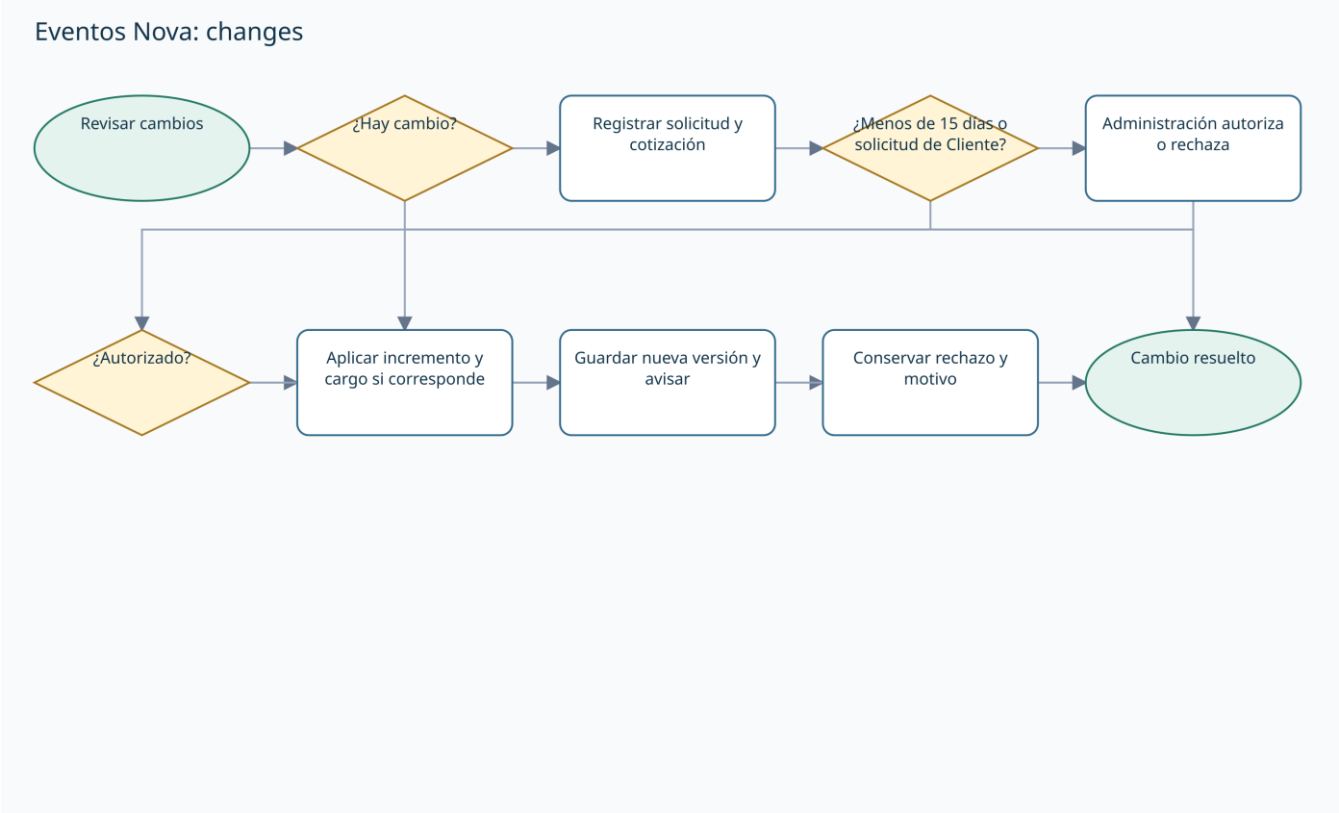
## BPMN - segmento 3 de 3



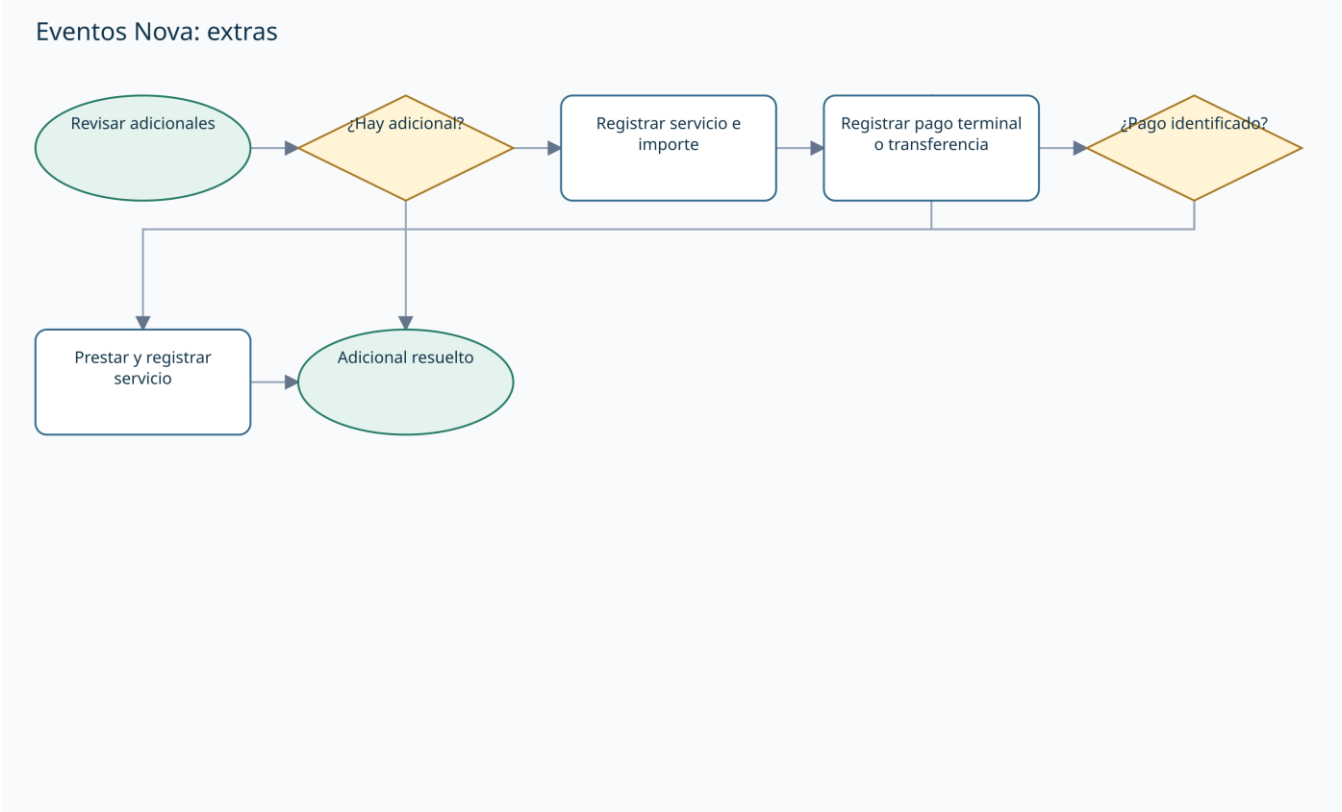

Fragmento del modelo BPMN del proceso integral.

El modelo contempla gateways, temporizador de 48 horas, rechazo de propuesta, contingencia, cambios y cierre.

## Subprocesos detallados




Gestión y autorización de cambios.




Servicios adicionales y condiciones de pago.

## 4. Glosario de dominio

Los términos 1-20 proceden del glosario U2. Se aclara su traducción al prototipo y se agregan términos de U3/PIA.

| N | Término | Definición | Código / alcance |
|---|---|---|---|
| 1 | Acta de cierre | Registro formal de conclusión con responsable, fecha y referencia. | Event.closure |
| 2 | Administración | Rol responsable de pagos y autorización comercial. | Principal.role ADMIN |
| 3 | Anticipo | 30% acumulado del monto firmado que habilita confirmación. | Event.pay |
| 4 | Backup de Respuesta Rápida | Proveedor activo marcado para contingencias. | supplier.backup |
| 5 | Cargo por urgencia | 15% del incremento cuando faltan menos de 15 días. | ChangePolicy |
| 6 | Cliente | Identidad dueña del evento y propuesta. | Event.client_id |
| 7 | Contrato | Referencia autenticada de firma y monto contratado. | Event.contract |
| 8 | Coordinador | Responsable operativo de propuesta y logística. | Principal.role COORDINADOR |
| 9 | Cotización / propuesta | Agregado con estado y versiones de servicios e importes. | Proposal |
| 10 | Ejecución del evento | Estado operativo del día programado. | EN_EJECUCION |
| 11 | Estado de proveedor | Pendiente, confirmado, rechazado, expirado o | SupplierRequest.state |

| N | Término | Definición | Código / alcance |
|---|---|---|---|
| | | cancelado. | |
| 12 | Fecha límite | 15 días naturales inclusive sin cargo. | ChangePolicy.free_days |
| 13 | Gateway de aprobación | Decisión BPMN que representa autorización. | Modelo BPMN, no entidad |
| 14 | Liquidación | Pago del saldo restante, antes o después de ejecución según H1. | Event.balance |
| 15 | Plataforma centralizada | API, persistencia y registro interno de avisos. | Sistema completo |
| 16 | Proveedor Categoría A | Clasificación de cumplimiento de la fuente, sin regla automatizada en este PIA. | Fuera del modelo funcional |
| 17 | Rentabilidad | Ingreso cotizado menos costo de proveedores confirmados vigentes. | profitability |
| 18 | Servicio adicional | Prestación añadida, registrada y pagada antes de entregarse. | Event.extras |
| 19 | Tipo de evento | Boda, graduación o empresarial; se conserva como descripción en esta demo. | title / descripción de catálogo |
| 20 | Versionado | Conservar V1.0, V1.1 y siguientes sin sobreescribir las anteriores. | Proposal.versions |
| 21 | Solicitud de cambio | Intención de modificar aforo, menú u horario | Event.changes |

| N | Término | Definición | Código / alcance |
|---|---|---|---|
| | | con impacto comercial. | |
| 22 | Cambio pendiente | Solicitud que espera decisión de Administración. | PENDIENTE |
| 23 | Solicitud de proveedor | Petición independiente con plazo continuo de 48 horas. | SupplierRequest |
| 24 | Aviso interno | Evento persistido con destinatarios, visible tras commit. | domain_events |
| 25 | Unidad de trabajo | Frontera transaccional que confirma o revierte cambios relacionados. | SqlUnitOfWork |
| 26 | Versión de concurrencia | Número usado para detectar escrituras basadas en datos obsoletos. | aggregates.version |

# 5. Arquitectura DDD y UML

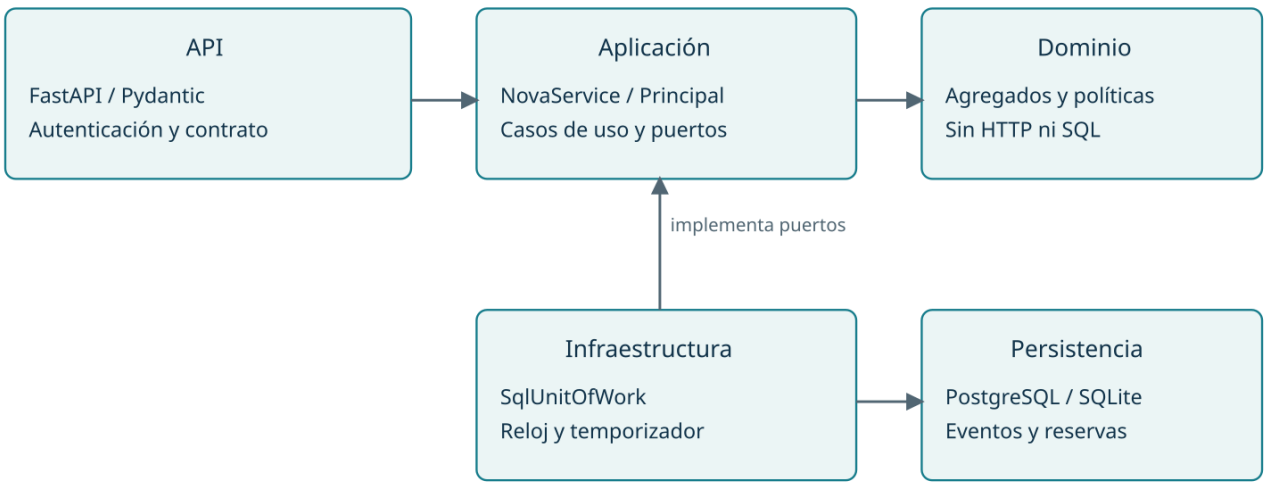
Los diagramas presentan el modelo de dominio y la arquitectura conceptual de Eventos Nova, elaborados conforme a los principios de Domain-Driven Design (DDD). La implementación en Python conserva las responsabilidades y reglas representadas, aunque determinados componentes se encuentran agrupados en servicios de aplicación o utilizan nombres diferentes. Esta correspondencia se documenta para mantener la trazabilidad entre el diseño arquitectónico y el código desarrollado.

## Continuidad con U3

Se mantienen los tres agregados del diseño: Propuesta, Evento y SolicitudDeProveedor. Los nombres Python son `Proposal`, `Event` y `SupplierRequest`. Se conserva el dominio independiente de FastAPI y SQLAlchemy, la unidad de trabajo y la concurrencia optimista. El PIA implementa también las extensiones US11-US15.

Las diferencias con el diseño teórico están registradas en ADR-006 a ADR-009. El documento describe el código existente, no clases pendientes como si estuvieran implementadas.

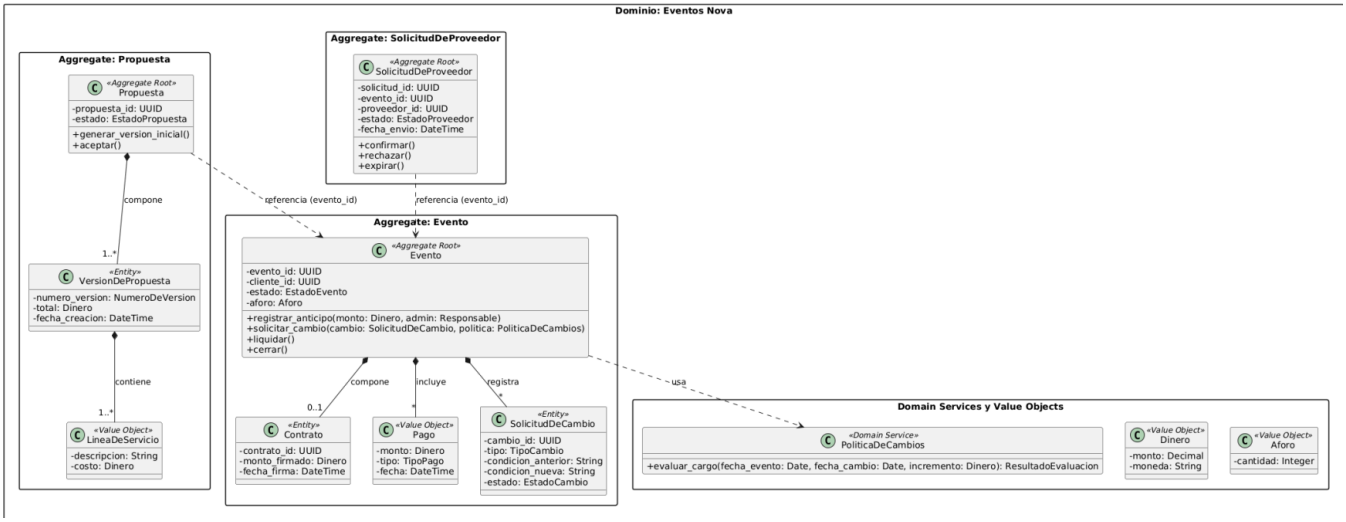
## Capas y dependencias




Arquitectura por capas y dependencias.

El punto de composición está en `src/api/main.py`. Los casos de uso reciben la fábrica de unidad de trabajo y el reloj. El dominio importa solo biblioteca estándar y módulos de su misma capa. El temporizador es un adaptador que invoca el caso de uso de vencimiento.

## DDD y fronteras




Modelo DDD y agregados de negocio.

El diagrama de clases del dominio representa la estructura de las principales entidades que intervienen en el funcionamiento de Eventos Nova. Su diseño se fundamenta en Domain-Driven Design (DDD), utilizando agregados, entidades, objetos de valor y servicios de dominio para organizar las reglas de negocio.

Se identifican tres agregados principales:

- Evento: representa la organización y administración de un evento. Contiene información sobre el cliente, estado y aforo. También administra contratos, pagos y solicitudes de cambio mediante operaciones como registrar anticipos, solicitar cambios, liquidar y cerrar eventos.

- Propuesta: administra las cotizaciones y sus versiones. Se relaciona con la entidad `VersionDePropuesta`, que conserva el número de versión, importe total y fecha de creación. Cada versión contiene las líneas de servicio correspondientes.

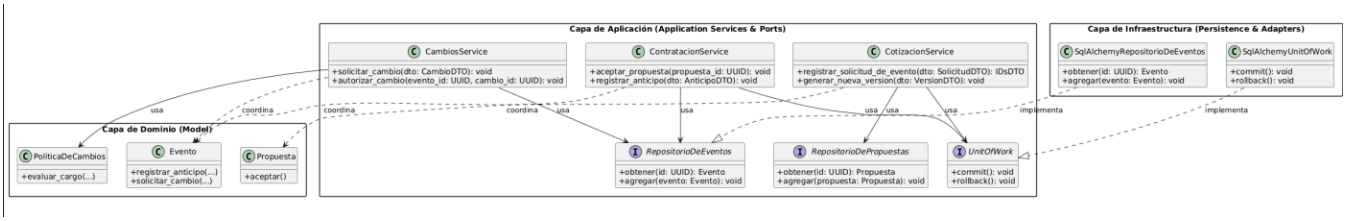
- SolicitudDeProveedor: representa las solicitudes realizadas a los proveedores. Almacena información del evento, proveedor, fecha de envío y estado de la solicitud. Sus operaciones permiten confirmar, rechazar o expirar solicitudes.

Adicionalmente, se incluyen los objetos de valor `Dinero` y `Aforo`, utilizados para representar importes monetarios y cantidades de asistentes.

El servicio de dominio `PoliticaDeCambios` concentra la evaluación de los cargos derivados de modificaciones en los eventos, considerando las fechas y los incrementos económicos.

La separación de estos elementos permite establecer responsabilidades claras, mantener la consistencia de los datos y facilitar el mantenimiento de las reglas del negocio.

## Diagrama de arquitectura por capas y dependencias




El segundo diagrama representa la organización del sistema en tres capas principales: dominio, aplicación e infraestructura. Esta separación permite mantener las reglas del negocio independientes de los mecanismos utilizados para almacenar y procesar la información.

Capa de dominio

Contiene los elementos fundamentales del negocio, como las entidades `Evento` y `Propuesta`, además del servicio `PoliticaDeCambios`.

Esta capa establece los comportamientos y restricciones que deben cumplirse durante la contratación, cotización y modificación de eventos.

Capa de aplicación

Coordina los casos de uso mediante los siguientes servicios:

- `CambiosService`: administra las solicitudes de modificación y su autorización.
- `ContratacionService`: coordina la aceptación de propuestas y el registro de anticipos.
- `CotizacionService`: permite registrar solicitudes de eventos y generar nuevas versiones de propuestas.

Estos servicios utilizan los elementos del dominio y se comunican con los mecanismos de persistencia a través de interfaces.

Capa de infraestructura

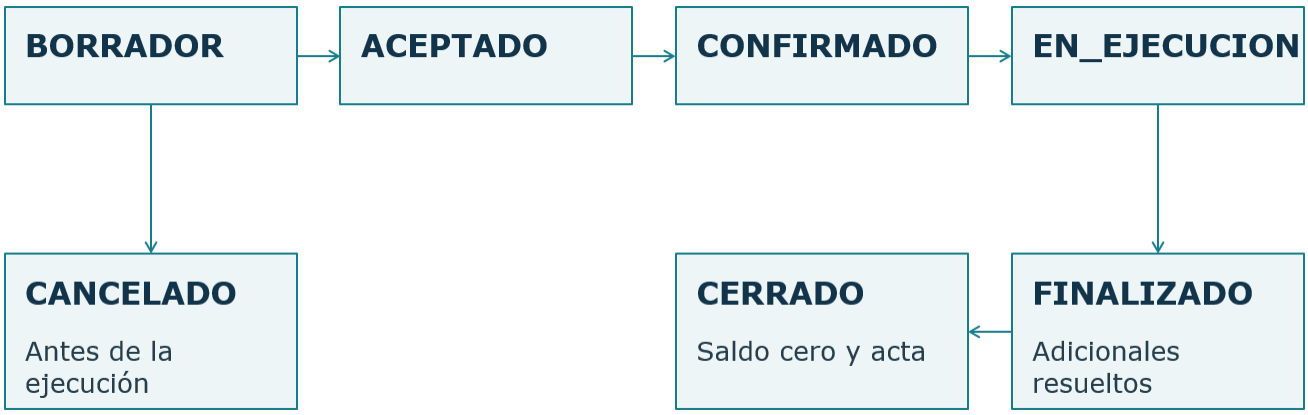
Proporciona las implementaciones necesarias para almacenar y recuperar la información.

Incluye `SqlAlchemyRepositorioDeEventos` y `SqlAlchemyUnitOfWork`, responsables del acceso a datos y de la coordinación de las transacciones.

Las interfaces `RepositorioDeEventos`, `RepositorioDePropuestas` y `UnitOfWork` permiten desacoplar los servicios de aplicación de las implementaciones concretas de infraestructura.

Esta arquitectura favorece la mantenibilidad, la aplicación de pruebas y el cumplimiento del principio de inversión de dependencias de SOLID.

## Estados e invariantes




### Estados de las entidades

En Eventos Nova, diferentes entidades mantienen estados que permiten identificar su situación dentro de los procesos del negocio.

El modelo contempla los siguientes tipos:

- `EstadoEvento`: identifica la situación actual de un evento durante su ciclo de vida.
- `EstadoPropuesta`: representa la situación de una propuesta comercial.
- `EstadoProveedor`: permite identificar la situación de una solicitud enviada a un proveedor.
- `EstadoCambio`: registra el estado de una modificación solicitada.

Estos estados permiten controlar las operaciones disponibles y evitar acciones que no correspondan con la situación actual de cada entidad.

### Invariantes del negocio

Las invariantes son condiciones que deben respetarse para mantener la consistencia del sistema.

Entre las principales reglas implementadas en Eventos Nova se encuentran:

1. Confirmación de eventos: la confirmación requiere cumplir las condiciones contractuales y registrar un anticipo acumulado de al menos el 30 % del importe correspondiente.
2. Control de proveedores: las solicitudes de disponibilidad tienen un plazo máximo de respuesta de 48 horas.
3. Gestión de cambios: las modificaciones realizadas a menos de 15 días del evento requieren evaluar cargos adicionales y aplicar las autorizaciones correspondientes.
4. Control de pagos: los importes registrados deben ser válidos y no deben producir inconsistencias financieras.
5. Historial de propuestas: las modificaciones deben conservar las versiones anteriores para permitir su consulta y trazabilidad.
6. Cierre de eventos: el cierre requiere cumplir las condiciones financieras y administrativas establecidas.

Estas restricciones se aplican mediante los métodos de las entidades, los servicios de dominio y las validaciones correspondientes.

Su finalidad es evitar operaciones incorrectas y garantizar que el sistema mantenga un comportamiento consistente.

## Specifications

`CAN_SEND = ActiveClient & HasLines & Authorized` implementa precondiciones de envío. `CAN_CLOSE = ~(PendingBalance | MissingInformation)` protege BR09. AND, OR y NOT intervienen en reglas reales. El contexto `Authorized=True` en el envío es consecuencia del permiso de Coordinación/Administración ya verificado. El flag CSV se usa solo en los casos diagnósticos.

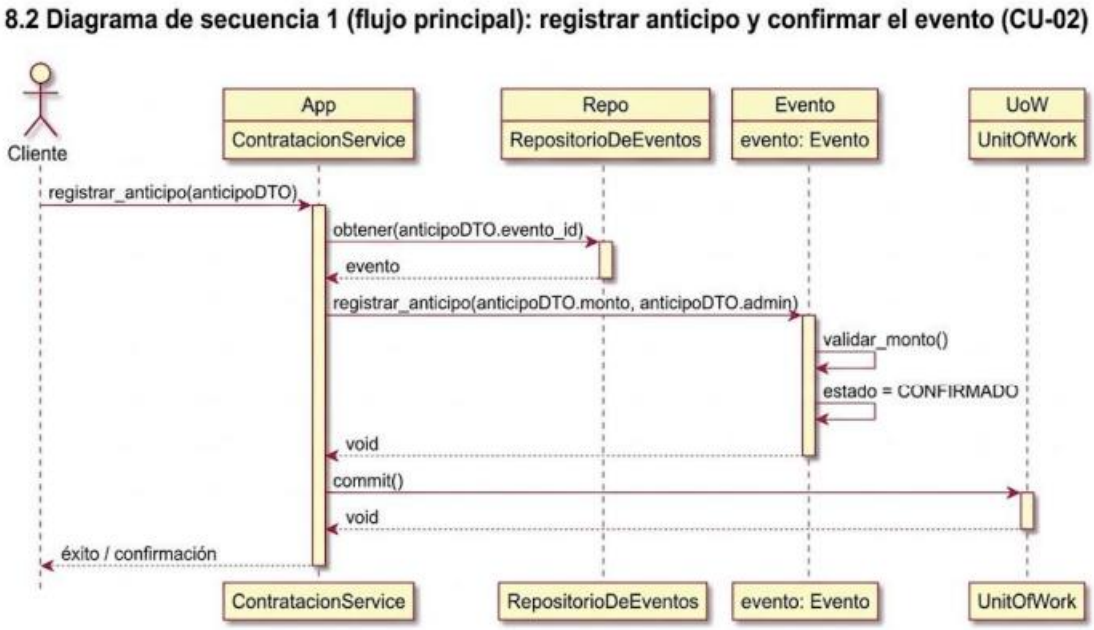
## Persistencia y eventos

SQLAlchemy Core crea tres tablas: `aggregates(kind,id,data,version)`, `domain_events(id,event_id,name,actor,at,payload)` y `supplier_bookings(supplier_id,day,request_id)`. Los agregados se guardan como JSON con identidad compuesta y versión. El formato conserva importes decimales como texto. Las referencias se validan en aplicación. No se afirma que exista una FK relacional dentro del JSON.

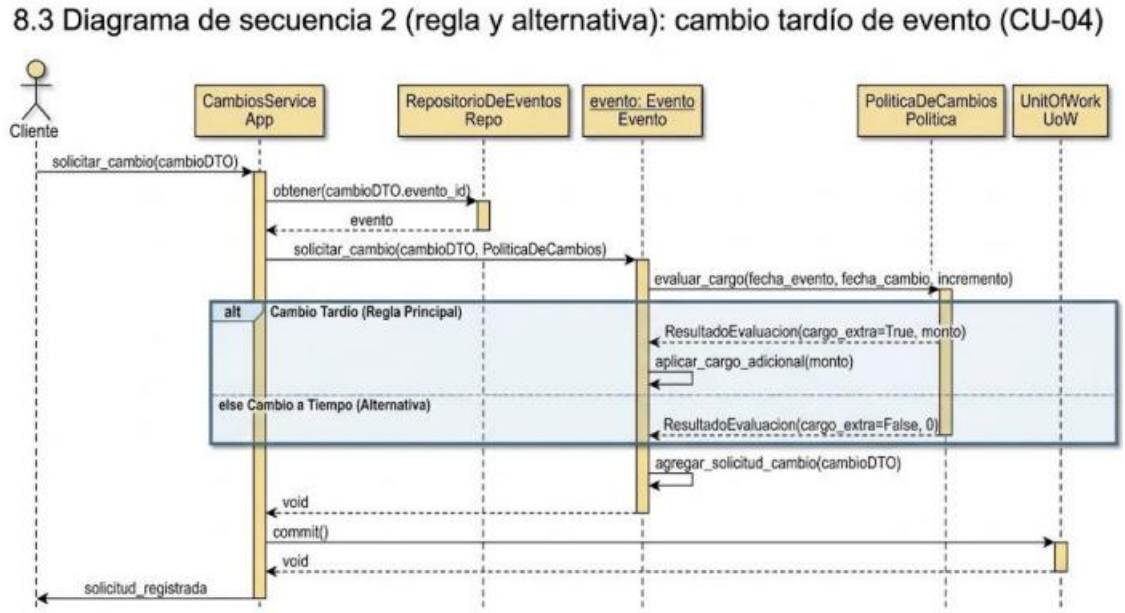
La escritura comprueba la versión previamente leída. Si otra transacción la modificó, responde 409 sin sobrescribirla. La clave única de proveedor/fecha impide doble reserva. La transacción revierte todos los cambios ante error. Alta y modificaciones de cotización actualizan Evento y Propuesta conjuntamente; aceptación también sincroniza ambos; respuesta/cancelación de proveedor sincroniza solicitud, costo y reserva.

Los eventos de dominio se registran atómicamente en `domain_events`, con nombre, responsable, fecha y payload. Un lector solo los ve tras commit. Constituyen auditoría y avisos internos. No hay broker, correo ni promesa de entrega externa. Si se añade correo, se necesita un consumidor con reintentos e idempotencia; esta entrega no lo simula.

## Secuencia de registrar anticipo y confirmar evento




## Secuencia de cambio tardío



## Patrones y SOLID

| Decisión | Aplicación comprobable | Consecuencia |
|---|---|---|
| Domain Model | Event.pay, SupplierRequest.respond, ChangePolicy.evaluate | Reglas comprobables sin HTTP ni DB |
| Specification | ActiveClient, HasLines, Authorized, PendingBalance, MissingInformation | Reglas combinables y comprobables por separado |
| State Machine | EVENT_TRANSITIONS y métodos de dominio | Cambios de estado explícitos, tabla completa probada |
| Repository/Unit of Work | SqlUnitOfWork.get/save/commit | Persistencia y transacciones fuera del dominio |
| Registro de Domain Events | emit dentro de la misma transacción | Auditoría consistente con cambios persistidos |
| CQRS básico | Métodos de lectura separados de execute | Una DB y snapshots compartidos, sin bus ni event sourcing |

SRP: dominio protege invariantes, aplicación coordina, infraestructura persiste, API valida contrato HTTP. NovaService agrupa casos de uso y es un punto que convendría separar por módulo si crece. OCP: las Specifications se componen y los relojes/adaptadores se inyectan; el registro de comandos aún exige editar la tabla de rutas. LSP: SqlUnitOfWork cumple los métodos del puerto y se prueba contra DB; el reloj fijo implementa el mismo contrato que SystemClock. ISP: Clock tiene solo now, UnitOfWork solo operaciones necesarias. DIP: dominio no depende de framework; aplicación recibe puertos y el arranque elige adaptadores.

## Límites operativos

Una instancia y un worker de Uvicorn para la demo. El temporizador registra vencimientos con hasta un intervalo adicional de 60 s, pero la aceptación a 48h exactas siempre se rechaza. No hay sistema de cuentas/login, OAuth, envío de correo, cobro bancario, certificado de firma o inventario multialmacén. Los tokens se administran por entorno. La publicación, ejecución real de Compose y el CI remoto deben comprobarse en el destino elegido, sin atribuirles evidencia local.

# 6. Decisiones técnicas y ADR

Los ADR-001 a ADR-005 documentan las decisiones arquitectónicas establecidas durante la Unidad 3. Los ADR-006 a ADR-009 incorporan los refinamientos técnicos adoptados durante el desarrollo del Producto Integrador de Aprendizaje, considerando la implementación, persistencia, seguridad, pruebas y despliegue del sistema.

## ADR-001 · Domain Model

Contexto: BR01-BR04, BR07 y BR09 combinan dinero, plazos y estados. Decisión: tres agregados con comportamiento. Alternativas: Transaction Script y Table Module. Consecuencias: mayor estructura inicial y pruebas de reglas independientes; evitar trasladar descuentos de las tiendas del laboratorio.

## ADR-002 · Frontera de Evento

Contexto: contrato, anticipo y saldo deben ser consistentes de inmediato. Decisión: pagos, cambios y adicionales dentro de Event; Propuesta y SolicitudDeProveedor separados por ID. Alternativa: cuenta separada con consistencia eventual. Consecuencias: Event crece y puede tener conflictos de escritura, mitigados mediante versiones.

## ADR-003 · Unidad de trabajo y eventos

Contexto: RNF08 y BR07 prohíben versiones contradictorias. Decisión U3: una transacción por agregado salvo alta/cambio de Evento y Propuesta; eventos visibles después del commit. Alternativa: consistencia eventual. Consecuencias: transacciones más amplias. El PIA extiende la atomicidad a aceptación y respuestas de proveedor con costo/reserva para no dejar estados parciales. Los avisos son registros internos transaccionales; no hay publicación a broker.

## ADR-004 · CQRS básico

Contexto: propuesta vigente, solicitudes, saldo y margen tienen consultas distintas. Decisión: lecturas y comandos separados en NovaService, una sola DB. Alternativa: event sourcing o bases separadas. Consecuencias: menor complejidad; algunas vistas reutilizan el agregado para calcular saldo, simplificación respecto al diseño U3 de lectores SQL separados.

## ADR-005 · H1-H5

Contexto: contradicciones U2 sobre día 15, autorización, base del cargo, liquidación y adicional. Decisión: seguir la tabla §1.4 del informe U3: 15 inclusive sin cargo, 15% sobre incremento, autorización administrativa tardía, liquidación previa o posterior, adicional con pago antes de prestación. Alternativa: bloquear avance hasta aclaración externa. Consecuencias: política explícita y localizable. El estado histórico del ADR es propuesta, mientras el informe marca esas resoluciones validadas por el equipo; no se presume aprobación del profesor.

## ADR-006 · Python, FastAPI y snapshots SQL

Contexto: U4 y U5 usan Python, FastAPI, pytest y SQLAlchemy. Decisión implementada: continuar esa tecnología, dominio dataclass, SQLAlchemy Core y JSON por agregado en PostgreSQL/SQLite. Alternativas: ORM con una tabla por entidad de U3 o reescritura en otro lenguaje. Consecuencias: puesta en marcha y transacciones simples, menos joins, restricciones referenciales internas en aplicación y necesidad de migrar explícitamente el formato JSON cuando evolucione. No existe una tabla por cada Value Object. PostgreSQL se configura en Compose/CI; SQLite se verifica localmente.

## ADR-007 · Seguridad y cotización comercial

Contexto: RNF03 requiere roles asociados a personas y el CSV solo trae una bandera genérica. Decisión implementada: tokens Bearer por identidad desde entorno, propiedad de cliente/proveedor, importes validados. Los cambios solicitados por Cliente esperan Administración; Coordinación aplica los no tardíos. El incremento se cotiza manualmente y se registra. Alternativas: confiar en un encabezado X-Role, aceptar roles en el body o implementar OAuth completo. Consecuencias: apropiado para demostración sin suplantación trivial del rol en el cuerpo; se necesita gestión de identidades y rotación de tokens para una operación comercial real.

## ADR-008 · Tiempo, estados y servicios adicionales

Contexto: BR02, BR03, BR08 y H1 necesitan fronteras precisas. Decisión implementada: instantes UTC, días naturales en America/Mexico_City, respuesta de proveedor estrictamente antes de 48h, inicio en la fecha del evento sin cambios pendientes, liquidación general flexible y cierre con saldo cero. El pago específico del adicional exige terminal/transferencia, aunque el pago general admite efectivo. Alternativa: usar fecha UTC para el límite o bloquear ejecución por deuda. Consecuencias: pruebas de medianoche y bordes; vencimiento revisado por temporizador con intervalo configurable y una sola instancia.

## ADR-009 · Evidencias y publicación

Contexto: la rúbrica exige CI/CD activo, Docker y URL funcional, pero preparar archivos localmente no demuestra ejecuciones remotas. Decisión implementada: generar evidencias reales de pytest/Ruff/API, workflow y despliegue configurables, e identificar pendientes de plataforma. Alternativa: incluir capturas o URLs de ejemplo como si fueran reales. Consecuencias: entrega revisable y reproducible; falta incorporar el repositorio final y comprobar Actions/Compose/producción en un destino real. El historial asistido no suplanta contribuciones del equipo.

# 7. Implementación y API REST

## Capas, código y responsabilidades

| Capa | Archivos | Función |
|---|---|---|
| Dominio | event.py, proposal.py, supplier.py, specifications.py, policies.py, state_machine.py | Invariantes, estados, dinero y reglas. |
| Aplicación | services.py, ports.py, security.py | Casos de uso, identidad, reloj y UoW. |
| Infraestructura | database.py, fixtures.py, scheduler.py | SQLAlchemy, transacciones, catálogos y plazos. |
| API | main.py, schemas.py | FastAPI, Pydantic, Bearer y OpenAPI. |

## Persistencia y consistencia

La tabla aggregates conserva snapshots de Evento, Propuesta y Solicitud de proveedor, con versión para concurrencia optimista. domain_events registra las auditorías y avisos internos; supplier_bookings impone la unicidad de proveedor por fecha. La unidad de trabajo realiza commit o rollback de cada operación. Los importes se guardan como texto decimal dentro de las estructuras.

## Contrato de endpoints

| Método | Ruta | Operación |
|---|---|---|
| GET | /catalogs/{kind} | Catalog |
| GET | /events | List Events |
| POST | /events | Create Event |
| GET | /events/{event_id} | Get Event |
| POST | /events/{event_id}/accept | Event Accept |
| POST | /events/{event_id}/approve | Event Approve |
| GET | /events/{event_id}/audit | Audit |
| POST | /events/{event_id}/cancel | Event Cancel |
| POST | /events/{event_id}/change | Event Change |
| POST | /events/{event_id}/close | Event Close |

| Método | Ruta | Operación |
|---|---|---|
| POST | /events/{event_id}/deliver_extra | Event Deliver Extra |
| POST | /events/{event_id}/extra | Event Extra |
| POST | /events/{event_id}/finish | Event Finish |
| POST | /events/{event_id}/pay | Event Pay |
| POST | /events/{event_id}/pay_extra | Event Pay Extra |
| GET | /events/{event_id}/profitability | Profitability |
| POST | /events/{event_id}/reject | Event Reject |
| POST | /events/{event_id}/reject_change | Event Reject Change |
| POST | /events/{event_id}/requests | Request Supplier |
| POST | /events/{event_id}/revise | Event Revise |
| POST | /events/{event_id}/send | Event Send |
| POST | /events/{event_id}/sign | Event Sign |
| POST | /events/{event_id}/start | Event Start |
| GET | /health | Health |
| GET | /me | Me |
| GET | /requests | Requests |
| POST | /requests/expire | Expire |
| POST | /requests/{request_id}/cancel | Cancel Supplier |
| POST | /requests/{request_id}/respond | Respond |

## Seguridad y manejo de errores

Un token Bearer identifica rol y propietario. La API responde 401 ante credenciales inválidas, 403 ante permisos insuficientes, 404 ante referencias inexistentes, 409 ante estados o versiones en conflicto y 422 ante violaciones de reglas o entradas inválidas. No se presenta OAuth ni pagos bancarios reales como funcionalidad existente.

## 8. Datos de referencia y adaptación CSV

Los seis CSV del paquete `16_Eventos_Nova/01_Alumno/datos` se conservan byte a byte en `data/original`. Contienen 40 actores, 20 recursos, 60 operaciones, 117 líneas, 120 casos y 20 transiciones. No se usaron respuestas reservadas al docente.

| Elemento fuente | Tratamiento | Justificación |
|---|---|---|
| actor_id y nombre Organizador | Referencia de Cliente de demo, conservando ID/nombre/activo/ciudad | El CSV usa actor genérico y no contiene cliente real |
| es_preferente, promocion_activa | Conservados en fuente, excluidos del dominio | El BRD Nova no define descuentos VIP |
| autorizado | Solo diagnóstico de Specifications | La API autentica por token, no por esta bandera |
| COTIZADO | BORRADOR de Evento y Propuesta V1.0 | U2/U3 exigen propuesta inicial, no flujo genérico CONTRATADO/REALIZADO |
| fecha_solicitud | Se conserva en la versión importada | No se presenta como fecha del evento |
| fecha de evento ausente | solicitud +60 días para cada fixture | Dato derivado, declarado, reproducible |
| costo proveedor ausente | Se registra al asignar, proveedores complementarios PROV-001/002/003 | No se infiere rentabilidad desde precio de venta |
| EVE-T001..060 | Precondiciones AND: activo, detalle y autorización, solo diagnóstico | No sustituye pruebas de API y permisos |
| EVE-B001..060, regla VIP | Adaptados a BR01 de contrato y anticipo del 30% | El paquete exige adaptar oráculos al BRD propio |
| EVE-S001..020 | COTIZADO se adapta a BORRADOR. aprobar=aceptar propuesta; completar=ciclo hasta cierre; cancelar=cancelación; repetir_aprobacion/saltar_estado= | La máquina real deriva de U2/U3 |

| Elemento fuente | Tratamiento | Justificación |
|---|---|---|
| | conflicto | |

En las transiciones `completar` el ensayo de dominio aporta contrato, pago total, fecha y proveedor confirmado sintéticos. Esto se declara en la matriz y no se confunde con un recorrido HTTP. Las pruebas de integración hacen el recorrido de API con persistencia real.

Los resultados esperados se calculan con un oráculo independiente en centavos enteros para el anticipo y tablas explícitas de estados. `scripts/evaluate_csv.py` usa el código de dominio para lo observado. Pytest parametriza y compara las 140 filas. Se exportan resultados reales/esperados, identificación de la adaptación y PASS/FAIL.

Los fixtures pueden conservar clientes inactivos o recursos no habilitados. El importador no borra esos casos adversos. Las altas por API aplican validación estricta de actividad, capacidades, cantidades y referencias.

> Los 120 casos y 20 transiciones del paquete CSV fueron comparados sin diferencias. Los 140 casos ya se incluyen en el recuento de pytest; no se suman dos veces.

# 9. Calidad y evidencia de pruebas

## Resultados reproducidos de forma independiente

| Prueba | Resultado | Alcance |
|---|---|---|
| pytest | 268 aprobadas, 2 omitidas, 0 fallidas | Sin DB PostgreSQL ni API externa en la revisión. |
| Cobertura líneas y ramas | 97.39% | Umbral requerido: 80%. |
| Ruff check y format | Aprobados | Código, scripts y pruebas. |
| Adaptación CSV | 120 + 20, sin fallos | Oráculos documentados. |

## Evidencia del pipeline del equipo

| Job | Resultado documentado | Evidencia |
|---|---|---|
| quality | 269 aprobadas, 1 omitida, 97.48% | Workflow y logs en Actions. |
| containers | 270 aprobadas, sin omisiones | PostgreSQL + App + Tests. |
| deploy | Omitido sin destino configurado | No certifica URL pública. |

Ejecución CI indicada en el repositorio: https://github.com/OmarHub-star/PIA-Eventos-Nova/actions/runs/42314931852 (commit 9fb4b061). La verificación local y la evidencia remota se identifican por separado para evitar afirmaciones no comprobadas.

## Casos críticos verificados

- Anticipo de 30%, contrato y rechazo de duplicados o sobrepagos.
- Cambios a 15 y 14 días con cargo y autorización según el plazo.
- Respuestas de proveedor a 48 horas exactas y vencimientos idempotentes.
- No duplicación de reserva de proveedor para una fecha.
- Rollback entre agregados y auditoría ante error.
- Aislamiento de roles, clientes, proveedores y finanzas.
- Adicionales pagados antes de prestación y cierre con saldo cero.
- Tabla completa de transiciones de la máquina de estados.

## Archivos de evidencia

| Ruta | Qué contiene |
|---|---|
| docs/evidencias/RESUMEN.md | Resultados, alcance y limitaciones. |
| docs/evidencias/resultados_120_casos.csv | Esperado y observado de casos. |
| docs/evidencias/resultados_20_transiciones.csv | Validación de transiciones. |
| docs/evidencias/demo_http.json | Recorrido HTTP documentado. |
| .github/workflows/ci.yml | Calidad, contenedores y despliegue condicional. |

## 9.1 Evidencias de funcionamiento del sistema desplegado

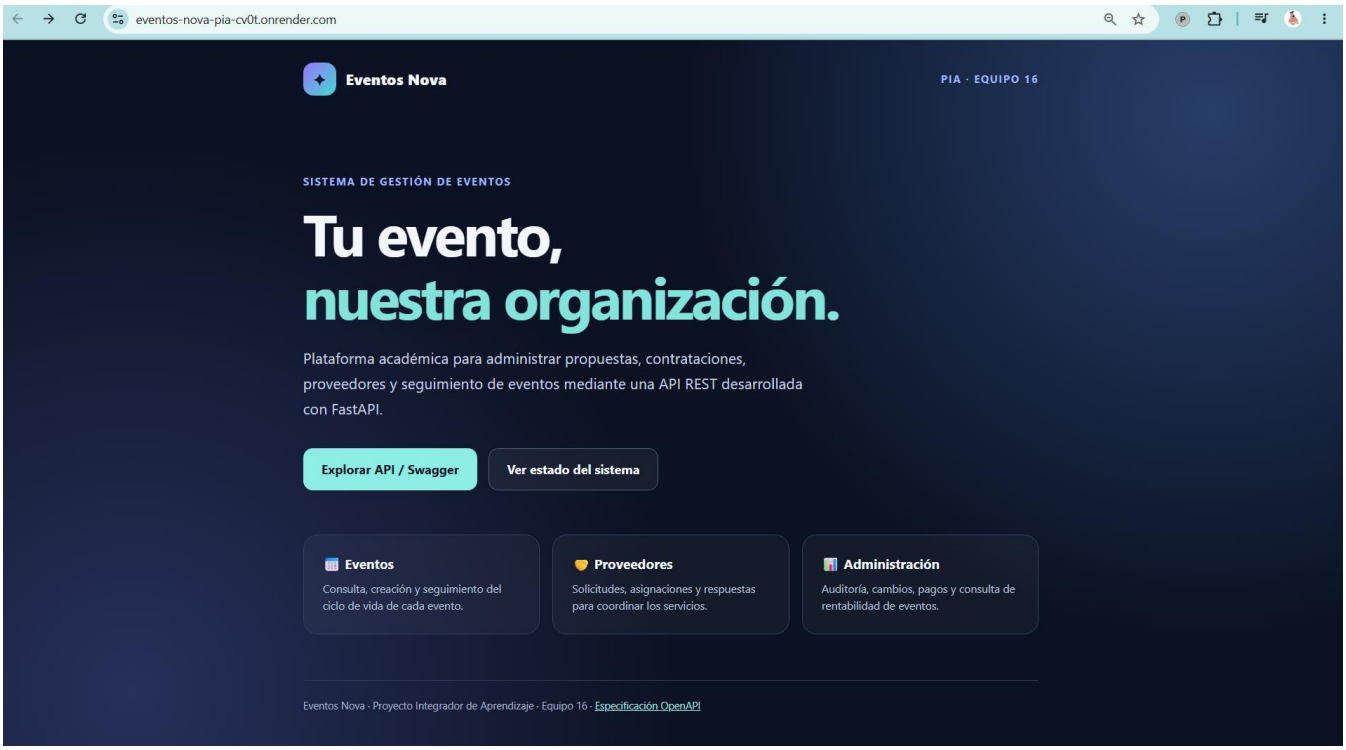



Figura 1. Servicio web Eventos Nova desplegado correctamente en Render.

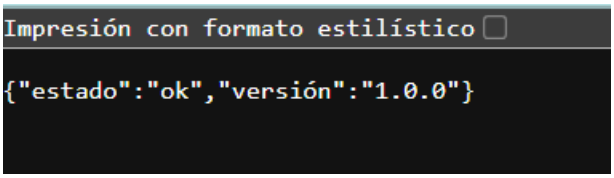



Figura 2. Verificación del estado operativo de la API mediante el endpoint /health.

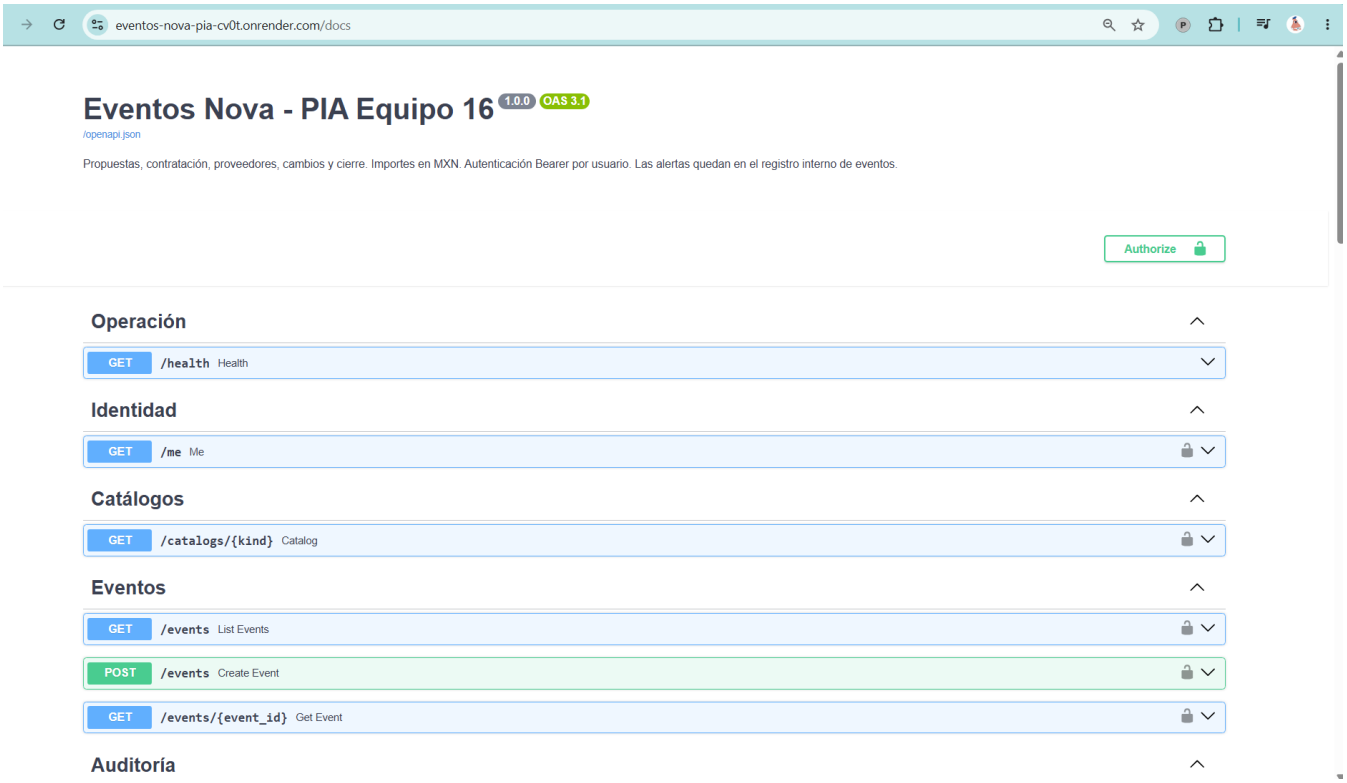



Figura 3. Documentación interactiva de la API REST mediante Swagger UI.

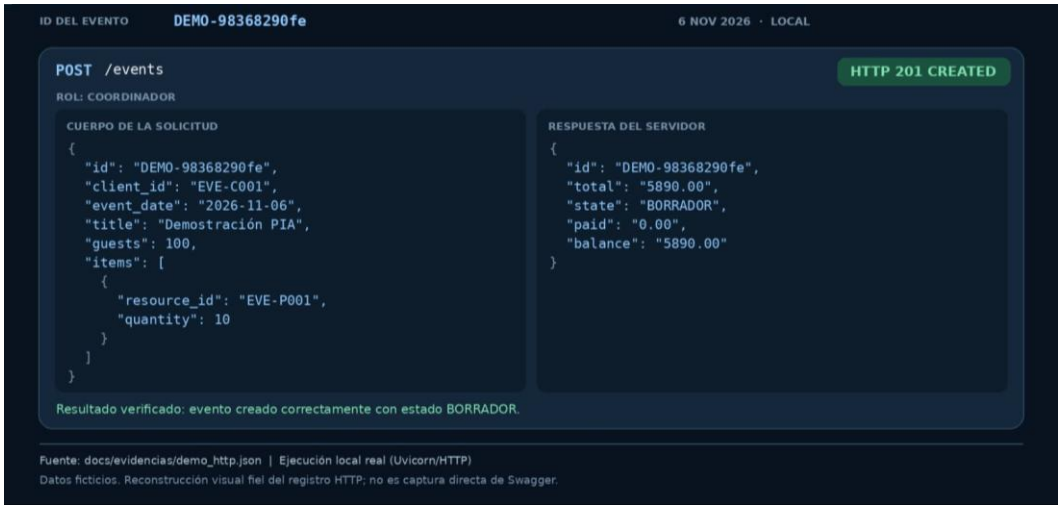


Figura 4. Registro exitoso de un evento mediante una solicitud HTTP.

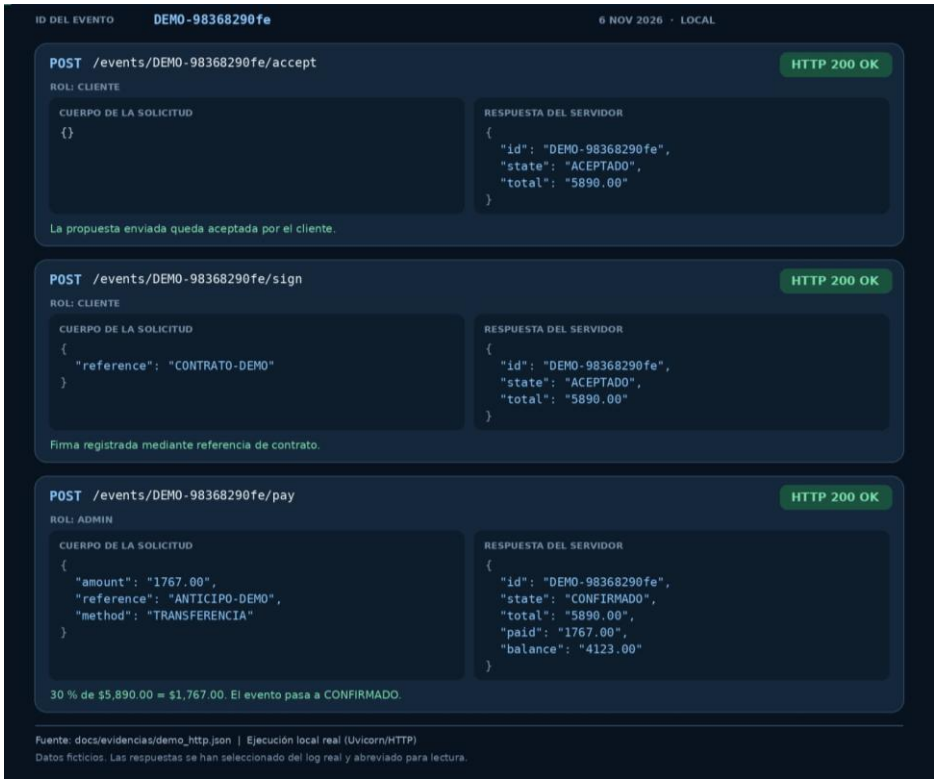



Figura 5. Validación del proceso de contratación y registro del anticipo.

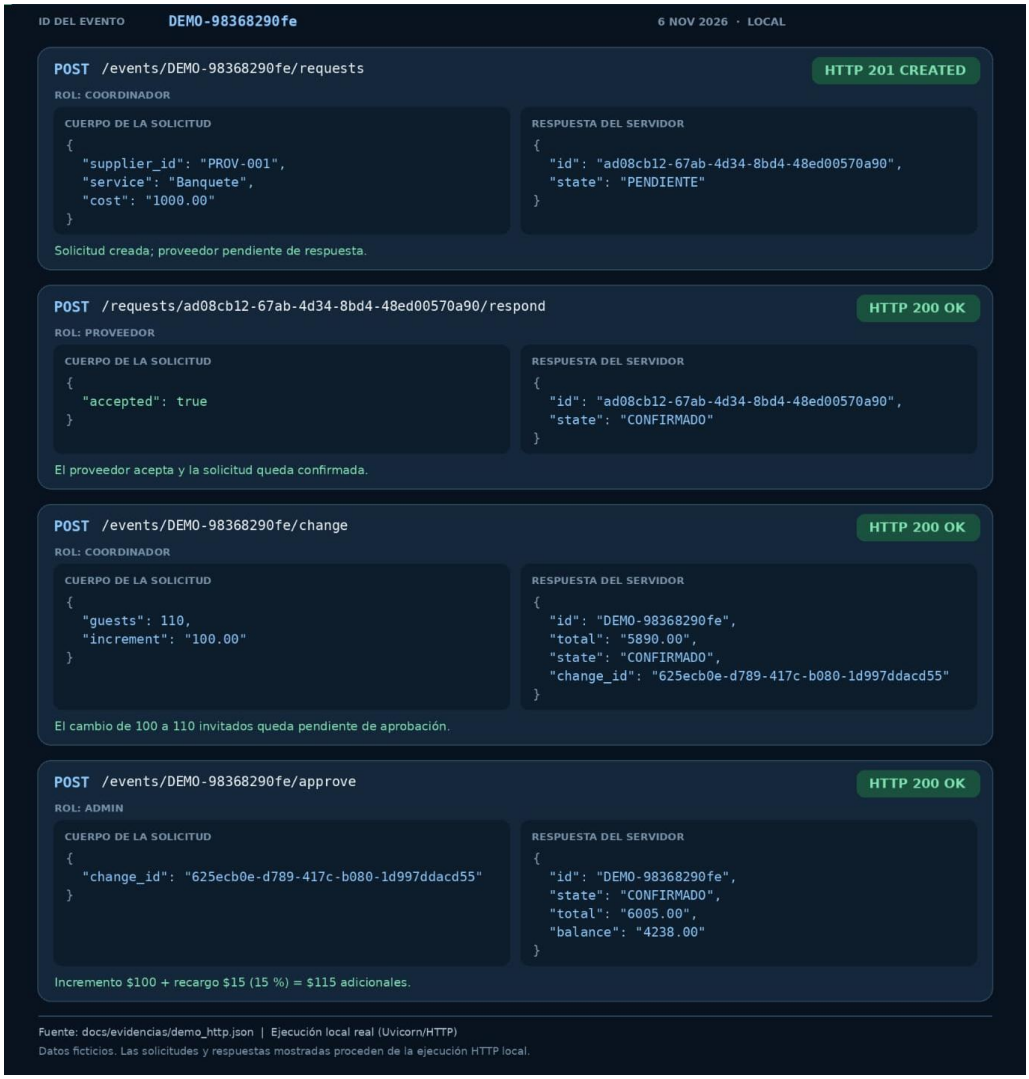



Figura 6. Ejecución de las reglas de negocio relacionadas con proveedores y cambios.

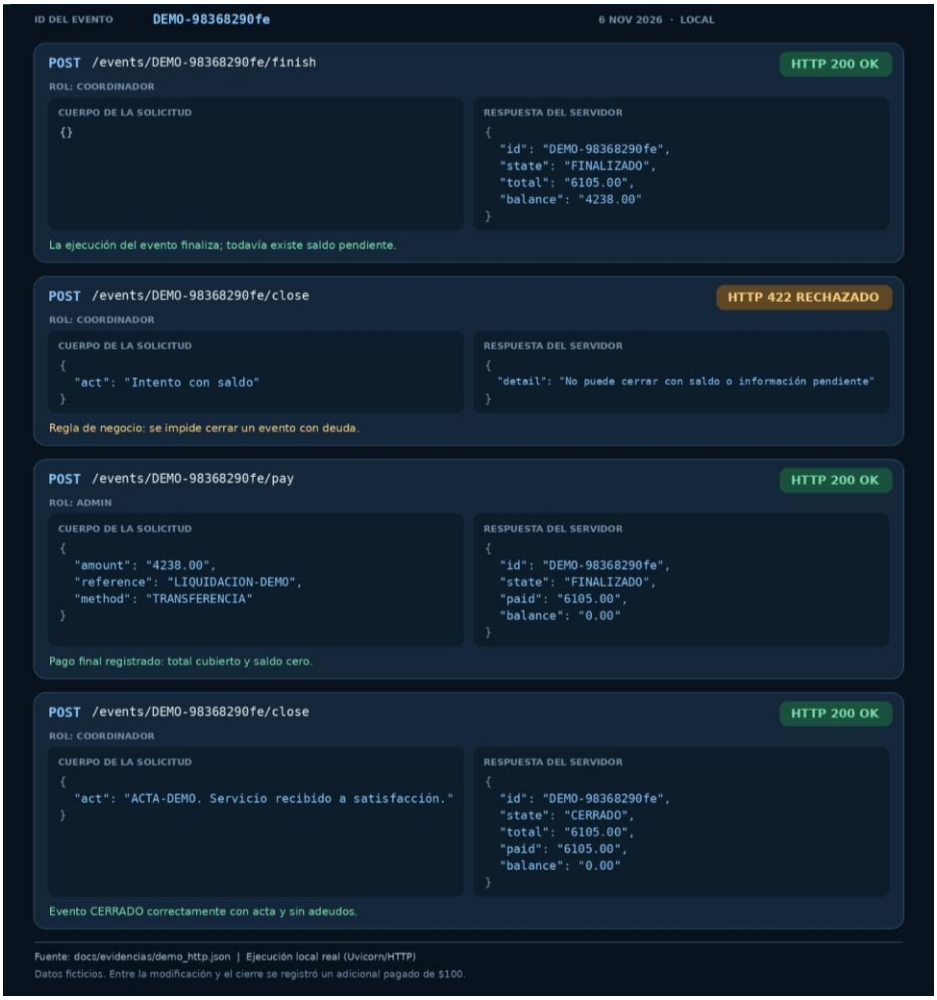



Figura 7. Finalización del proceso y cierre de un evento.

# 10. Docker, CI/CD y despliegue

## Ejecución reproducible

El Dockerfile usa Python 3.12 y un usuario no privilegiado. Docker Compose define db con PostgreSQL 16, app con FastAPI y tests con perfil quality; los servicios tienen verificaciones de salud y el volumen nova_db conserva los datos. .env.example separa configuración y secretos del código.

```
python scripts/init_env.py
docker compose up --build -d db app
docker compose --profile quality run --rm tests
```

## CI y despliegue continuo

GitHub Actions ejecuta Ruff, pytest, cobertura y evaluación CSV. Un segundo job construye contenedores y ejecuta pruebas con PostgreSQL. El job deploy depende de ambos y requiere configurar RENDER_DEPLOY_HOOK; sin ese secreto no solicita publicación.

## Estado de producción al corte

| Componente | Estado |
|---|---|
| render.yaml | Archivo de configuración preparado para el despliegue de la aplicación en Render. |
| Conexión repositorio/Render | Aplicación desplegada y disponible públicamente en Render |
| URL pública | https://eventos-nova-pia-cv0t.onrender.com/ |
| /health, /docs, /openapi.json públicos | Los endpoints /health, /docs y /openapi.json fueron verificados y responden correctamente. |

# 11. Matriz de trazabilidad

## US - regla - endpoint - prueba

| US | Reglas | Operación | Prueba |
|---|---|---|---|
| US01 | BN02, BR07 | /events | US07_revision_history_and_duplicate_accept |
| US02 | BR07 | /events/{id}/accept | US07_revision_history_and_duplicate_accept |
| US03 | BR01 | /events/{id}/pay | BR01_30_percent_boundary |
| US04 | BN03, BN06 | /events/{id}/requests | US01_US15_full_business_cycle |
| US05 | BR02 | /requests/{id}/respond | BR02_continuous_hours |
| US06 | BR02, RNF07 | Temporizador y /requests/expire | US06_48_hours_and_expiration_idempotence |
| US07 | BR07, BR10 | /events/{id}/revise | US07_revision_history_and_duplicate_accept |
| US08 | RNF07, RNF08 | /events/{id}/change y /events/{id}/audit | US08_US10_changes_authorization_and_notification |
| US09 | BR03 | /events/{id}/change | client_change_waits_for_commercial_review |
| US10 | BR04 | /events/{id}/approve y reject_change | US08_US10_changes_authorization_and_notification |
| US11 | BR06, BN08 | /requests/{id}/cancel y /events/{id}/requests | US11_backup_replacement_and_booking |
| US12 | BN05 | /events/{id}/profitability | US01_US15_full_business_cycle |
| US13 | BR05, BR08 | /events/{id}/extra, pay_extra, deliver_extra | BR05_BR08_extra_requires_payment_and_trace |
| US14 | BR09, H1 U3 | /events/{id}/pay | post_event_liquidation_follows_U3_H1 |
| US15 | BR09, BR10 | /events/{id}/close | US01_US15_full_business_cycle |

## Correspondencia con la rúbrica

| Bloque | Evidencia | Observación |
|---|---|---|
| Análisis (25%) | BRD, US01-US15, Gherkin, BPMN, glosario | Trazados a reglas y código. |
| Arquitectura (25%) | DDD/UML, Specification, State Machine, SOLID, ADR | Implementación consistente. |
| Implementación (25%) | API REST, SQL, UoW, casos principales | Suite local aprobada. |
| Calidad/defensa (25%) | Cobertura, CI, Docker, demo | Despliegue público realizado en Render. Endpoints de salud y documentación verificados. Pruebas automatizadas y configuración de CI/CD documentadas. |

# 12. Guía de demostración y defensa

## Preparación

Usa una base ficticia recién inicializada. Inicia la aplicación y abre `/docs`. Cambia el token de Authorize según el rol indicado. Elige un ID nuevo, por ejemplo DEMO-PIA-01. El proveedor de demo solo puede aceptar un evento por fecha; para repetir el recorrido completo prepara otra base de demo o usa una fecha y proveedor disponibles.

## Recorrido en vivo de 8 a 12 minutos

- 1. **Coordinación:** `POST /events`, cliente EVE-C001, fecha de hoy en México, 100 invitados, recurso EVE-P001 y cantidad 10. Total 5890.00 MXN, versión V1.0.
- 2. **Coordinación:** `send` con `{}`. **Cliente:** `accept` con `{}`. Repetir accept demuestra 409.
- 3. **Cliente:** `sign`, referencia CONTRATO-01. **Administración:** `pay`, monto 1767.00, referencia ANTICIPO-01, método TRANSFERENCIA. Se confirma el evento.
- 4. **Coordinación:** crear solicitud en `/events/{id}/requests`: proveedor PROV-001, servicio Banquete, costo 1000.00. **Proveedor PROV-001:** responder accepted=true en `/requests/{request_id}/respond`.
- 5. **Coordinación:** `change` con guests=110 e increment=100.00. Al ser hoy, queda pendiente con cargo 15.00. **Administración:** `approve` con change_id. Total 6005.00, nueva versión conservando V1.0. Consultar audit muestra condición anterior, nueva y destinatarios.
- 6. **Coordinación:** `start`. Registrar `extra`, descripción Música, importe 100.00. Intentar `deliver_extra` sin pago demuestra 422. **Administración:** `pay_extra`, extra_id, referencia EXTRA-01 y método TERMINAL. **Coordinación:** `deliver_extra` y `finish`.
- 7. **Coordinación:** intentar `close` demuestra bloqueo por saldo. **Administración:** pagar el saldo informado por la API. **Coordinación:** cerrar con acta ACTA-01. Estado CERRADO, saldo 0.00.
- 8. **Administración:** consultar profitability. Ingreso 6105.00, costo proveedor 1000.00, margen 5105.00 MXN. Explicar que margen cotizado y cobro acumulado son conceptos distintos.

El script `python scripts/demo_http.py --base-url http://localhost:8000` realiza el mismo recorrido y guarda evidencia sin tokens. Requiere identidades configuradas para EVE-C001 y PROV-001. Los importes del ejemplo dependen del CSV original incluido.

## Excepciones adicionales

- Cambio a 15 días exactos: cargo cero. A 14 días: 15% del incremento.
- Proveedor que cancela: devolver Backup activo y registrar nueva solicitud con `replaces`.
- 48h: mostrar test determinista del borde sin esperar 48 horas en clase. El temporizador real revisa vencimientos cada minuto.
- Rollback: `test_failed_change_rolls_back_event_proposal_and_notifications` comprueba fallo entre agregados sin dejar notificación.
- Seguridad: token de Cliente no puede registrar pagos ni consultar margen. Otro Cliente no accede al evento.

### Preguntas para la defensa

**¿Por qué DDD?** Las reglas combinan estado, dinero y tiempo. Event conserva pagos/contrato/cambios; Proposal conserva historial; SupplierRequest tiene un plazo independiente.

**¿Dónde están AND, OR y NOT?** CAN_SEND combina activo, detalle y autorización. CAN_CLOSE niega deuda o información faltante. Hay pruebas de cada combinación.

**¿Cómo evitan errores monetarios?** Decimal y redondeo ROUND_HALF_UP a centavos. JSON y DB conservan importes como texto. Nunca se calcula dinero con float.

**¿Qué sucede a 15 días o a 48h exactas?** Día 15 no genera cargo por BR03/H2. A 48h ya no se acepta respuesta por el límite estricto de U3/BPMN.

**¿Qué protege la transacción?** Agregados, costo/reserva y eventos de dominio relacionados. Si algo falla, rollback. Un update condicionado por versión evita perder cambios de otra transacción.

**¿Qué cambió frente a U3?** SQLAlchemy Core con snapshots JSON, un servicio coordinador, vistas sencillas y avisos internos persistidos. Se implementaron extensiones de adicionales y cierre. Los ADR documentan costos y límites.

**¿Son reales las métricas?** Se generan con pytest-cov y se adjunta JUnit/log/HTML. Las metas comerciales del BRD no se presentan como mejoras medidas.

**¿Qué falta verificar fuera de este entorno?** El servicio ya cuenta con un despliegue público en Render y dispone de documentación Swagger/OpenAPI accesible. Como parte de la preparación para la defensa, se debe confirmar el funcionamiento de los casos de uso autenticados en producción, comprobar que la configuración de seguridad corresponda al entorno desplegado y disponer de evidencias actualizadas de Docker Compose y del pipeline CI/CD.

## 13. Conclusiones

### Resultados

La solución transforma reglas de Eventos Nova en comportamientos observables y verificables. El contrato y el anticipo gobiernan la confirmación; los proveedores tienen plazos y restricciones de reserva; los cambios mantienen historial y efectos económicos; el cierre exige obligaciones financieras resueltas.

Los agregados están separados del framework y de la base de datos. La trazabilidad vincula 15 historias a endpoints y pruebas, y el porcentaje de cobertura local supera el requisito del 80%. La evidencia documentada de CI complementa las pruebas en SQLite con PostgreSQL y Docker.

### Limitaciones transparentes

- La autenticación utiliza tokens por configuración; no ofrece registro u OAuth.
- Las notificaciones son internas, no correo ni mensajería externa.
- Los pagos son referencias y registros, no transacciones bancarias.

- La persistencia usa snapshots JSON y creación de tablas sin migraciones productivas.
- El vencimiento automático está diseñado para una instancia de demostración.
- El sistema dispone de un despliegue público con documentación OpenAPI; la demostración de los procesos autenticados requiere mantener correctamente configurados los tokens y permisos del entorno de producción

# 14. Fuentes, anexos y verificación final

## Fuentes utilizadas

- Rúbrica oficial PIA_CLN_Proyecto_Integrador_AD2026.pdf, noviembre de 2026.
- BRD, historias, glosario y BPMN de U2, conservados en docs/fuentes.
- Informe de arquitectura, diagramas y ADR de U3, en docs/architecture.
- Repositorio de Eventos Nova: https://github.com/OmarHub-star/PIA-Eventos-Nova
- OpenAPI versionado: docs/api/openapi.json; Swagger en /docs.
- Resultados y trazabilidad: docs/evidencias/RESUMEN.md y docs/TRAZABILIDAD.csv.
- Documentación FastAPI: https://fastapi.tiangolo.com/
- Documentación SQLAlchemy: https://docs.sqlalchemy.org/en/20/
- Documentación Docker Compose: https://docs.docker.com/compose/
- Documentación Render: https://render.com/docs/